



UNIVERSITAT DE
BARCELONA

Facultat de Matemàtiques i Informàtica
Universitat de Barcelona

Manual de Introducció al Anàlisis Forense en entorns Linux

Documento complementario al Trabajo de Fin de Grado:

**ForensicXM: Una herramienta nativa de análisis forense
para sistemas GNU/Linux**

Autor: Xavier Marzo Pelaz

Director: Dr. Raul Roca Canovas

Realitzat a: Departament de Matemàtiques i Informàtica

Barcelona, 10 de junio de 2026

Índice general

1. Fundamentos de GNU/Linux	5
1.1. El origen de Linux	5
1.1.1. Distinción Fundamental: Kernel vs SO	5
1.1.2. El Proyecto GNU y la Fusión con Linux	5
1.1.3. La Filosofía del Software Libre	5
1.2. Arquitectura y Tipología de las Distribuciones GNU/Linux	6
1.2.1. Definición y Componentes de una Distribución	6
1.2.2. Características Clave y Relevancia Forense	6
1.2.3. El árbol genealógico y las relaciones entre distros	6
1.3. WSL como Entorno Integrado para Análisis Forense	7
1.3.1. Evolución y Arquitectura de WSL	7
1.3.2. Proceso de Instalación Simplificado	7
1.3.3. Implicaciones Forenses Críticas de WSL 2	7
1.3.4. Archivos de Configuración Avanzados para la Investigación	8
1.3.5. Artefactos del Host de Windows	8
1.4. Appliances y Contenedores Linux	8
1.4.1. Appliances de Linux: Especialización y Retos Forenses	8
1.4.2. Informática Forense de Contenedores (Docker)	8
1.5. Estructura de Directorios	9
1.6. Permisos de Archivo y Directorio	11
1.6.1. Estructura de permisos	11
1.6.2. Modificación de permisos (chmod)	11
1.6.3. Permisos especiales	12
1.6.4. Indicadores forenses	12
1.7. Gestión de Usuarios y Grupos	13
1.7.1. Identificadores y tipos de Usuario	13
1.7.2. Archivos críticos para la investigación forense	13
1.7.3. Archivos de Grupos	13
1.7.4. Utilidades de Gestión	14
1.8. Interfaz de línea de comandos (Shells) y Artefactos de Historial	14
1.8.1. Tipos de Shells y Archivos de Historial	14
1.8.2. Archivos de configuración críticos (bash)	14
1.8.3. Técnicas Anti-forenses	15
2. Registro de Eventos, Logs y Monitorización	16
2.1. Autenticación y Seguridad	16
2.1.1. Fundamentos y Marco de Autenticación (PAM)	16
2.1.2. Logs Clave de Autenticación Basados en Texto	17

2.1.3.	Logs Binarios y Herramientas de Análisis Forense	18
2.2.	Syslog y Kernel	19
2.3.	Servidores Web	22
2.4.	Firewalls and Proxies	24
2.5.	Auditd	25
2.5.1.	Sistemas Vivos (Live Systems)	26
2.5.2.	Forensia Deadbox (Logs Recolectados)	26
2.6.	Sysmon para Linux	27
2.7.	VMware ESXi and vCenter Log Forensics	28
2.8.	Miscellaneous	30
3.	Sistemas de Archivos en Linux	33
3.1.	El Sistema de Archivos EXT2	33
3.1.1.	Contexto y Características Fundamentales de EXT2	33
3.1.2.	Estructura Lógica y Metadatos (Valor Forense)	33
3.1.3.	Marcas de Tiempo: La Cronología Forense	33
3.1.4.	Herramientas Forenses Esenciales para EXT2	34
3.2.	El Sistema de Archivos EXT3	34
3.2.1.	Importancia del Journaling en Forense	35
3.2.2.	Modos de Operación del Journaling en ext3	35
3.2.3.	Metadatos y Límites del Sistema	35
3.2.4.	Herramientas Forenses Fundamentales	35
3.2.5.	Desafío Principal del Análisis en ext3	36
3.3.	El Sistema de Archivos EXT4	36
3.3.1.	Características de ext4 fundamentales para el análisis forense	36
3.3.2.	Herramientas clave para la investigación en ext4	37
3.3.3.	Comandos TSK fundamentales para ext4	37
3.4.	Análisis de Sistemas de Archivos	37
3.5.	Timestomping	40
3.6.	BTRFS	41
3.6.1.	Características Centrales y Diseño	41
3.6.2.	Metadatos Forenses y O Time	42
3.6.3.	Herramientas Específicas para la Investigación Forense	42
3.7.	El Sistema de Archivos XFS	43
3.8.	Sistema de Archivos OpenZFS	44
3.8.1.	Características Arquitectónicas y de Rendimiento	44
3.8.2.	Integridad de Datos como Característica Principal	45
3.8.3.	Manejo de Transacciones y Timestamps	45
3.8.4.	Uso y Comandos Comunes	45
3.8.5.	Contexto en el Ámbito Digital Forense	45
4.	Técnicas de Persistencia de Malware	46
4.1.	Persistencia de Malware y Gestión de Servicios: init.d vs systemd	46
4.1.1.	Persistencia mediante init.d (SysV Init)	46
4.1.2.	Persistencia mediante systemd	46
4.1.3.	Análisis Comparativo en Forense	47
4.2.	Persistencia mediante Programación de Tareas: Systemd y Cron	47
4.2.1.	Temporizadores Systemd (Systemd Timers)	47

4.2.2.	Tareas Cron (Cron Jobs)	48
4.2.3.	Artefactos Forenses y Ubicación	48
4.3.	Persistencia mediante Claves SSH	48
4.3.1.	Generación de Credenciales (Lado del Atacante)	48
4.3.2.	Implantación de la Clave (Lado de la Víctima)	49
4.3.3.	Ejecución y Verificación	49
4.4.	Mecanismos de Persistencia Avanzada en Sistemas Linux	49
4.4.1.	Manipulación de la Shell de Usuario (<code>.bashrc</code> y <code>.bash_profile</code>)	49
4.4.2.	Ejecución en el Inicio del Sistema (<code>/etc/rc.local</code>)	50
4.4.3.	Persistencia Basada en Eventos de Hardware (<code>udev rules</code>)	50
4.4.4.	Ejecución en Entornos Gráficos (XDG Autostart)	50
4.4.5.	Scripts de Eventos de Red (NetworkManager)	50
4.4.6.	Gestión de Cuentas y Privilegios	50
5.	Adquisición de Evidencia Digital	51
5.1.	Adquisición de Evidencia Digital: Evolución de Herramientas	51
5.1.1.	La herramienta fundacional: <code>dd</code> (1974)	51
5.1.2.	Evolución para la integridad: <code>dcfldd</code> (2002)	51
5.1.3.	Especialización Forense: <code>dc3dd</code> (2008)	52
5.2.	Captura de Memoria Volátil en Linux: AVML	52
5.2.1.	Contexto Histórico y Limitaciones	52
5.2.2.	AVML (Acquire Volatile Memory for Linux)	53
5.2.3.	Procedimiento Técnico	53
5.3.	UAC (Unix-like Artifacts Collector)	53
5.3.1.	Metodología y Arquitectura	53
5.3.2.	Perfiles de Recolección	54
5.3.3.	Estructura de la Evidencia Recolectada	54
5.4.	Adquisición de Evidencia Digital en Entornos Virtualizados y Cloud	55
5.4.1.	Plataformas de Virtualización Local (On-Premise)	55
5.4.2.	Entornos Cloud (Azure y AWS)	56
5.4.3.	Estrategias de Contingencia y Análisis en Vivo	56
6.	Líneas de Tiempo Forenses	57
6.1.	Creación y Análisis de Líneas de Tiempo Forenses	57
6.1.1.	Generación del Archivo Intermedio (<code>fls</code>)	57
6.1.2.	Procesamiento y Normalización (<code>mactime</code>)	57
6.1.3.	Interpretación de Datos (Timeline Explorer)	58
6.1.4.	Flujo de Trabajo Analítico	58
6.2.	Plaso: Definición y Arquitectura	58
6.2.1.	Componentes Principales y Flujo de Trabajo	58
6.2.2.	Metodologías de Filtrado y Recolección	59
6.2.3.	Análisis Contextual y Correlación	59
6.2.4.	Herramientas Complementarias	60
7.	Análisis Forense de Memoria con Volatility	61
7.1.	Enumeración y Análisis de Procesos en Forense de Memoria (Linux)	61
7.1.1.	Introducción al Análisis de Memoria	61
7.1.2.	Herramientas de Análisis: Volatility 3	61
7.1.3.	Métodos de Enumeración de Procesos	61

7.1.4.	Análisis de Contexto y Recursos del Proceso	61
7.1.5.	Metodología de Investigación Recomendada	62
7.2.	Recuperación de Historial de Comandos con Volatility 3	62
7.2.1.	Introducción al Plugin <code>linux.bash</code>	62
7.2.2.	Ejecución y Gestión de Datos Temporales	63
7.2.3.	Correlación de Eventos e Indicadores de Compromiso (IoC)	63
7.2.4.	Opciones de Filtrado y Salida Profesional	63
7.3.	Análisis de Actividad de Red en Memoria Linux	64
7.3.1.	El Plugin <code>linux.sockstat</code> : Más allá de <code>lsof</code>	64
7.3.2.	Metodología de Análisis y Filtrado	64
7.3.3.	Automatización y Enriquecimiento con Herramientas Auxiliares	64
7.3.4.	Conclusión Forense	64
7.4.	Detección de Inyección de Código con <code>linux.malfind</code>	65
7.4.1.	Propósito y Funcionamiento	65
7.4.2.	Análisis de la Salida (Output)	65
7.4.3.	El Desafío de los Falsos Positivos	65
7.4.4.	Metodología de Validación Avanzada	66
7.5.	Extracción de Datos Volátiles y Volcado de Memoria	66
7.5.1.	Consideraciones Previas: La Dependencia de la RAM Física	66
7.5.2.	Herramientas de Extracción (Argumento <code>--dump</code>)	66
7.5.3.	Tabla Resumen de Metodología de Volcado	67
7.5.4.	Flujo de Trabajo Post-Extracción	68
7.6.	Análisis Avanzado: Módulos del Kernel, Entorno y Escaneo Yara	68
7.6.1.	Inspección del Kernel con <code>linux.lsmmod</code>	68
7.6.2.	Contexto de Ejecución: <code>linux.invars</code> (o <code>envirs</code>)	68
7.6.3.	Escaneo de Memoria Virtual: <code>linux.vma_yarascan</code>	69
7.6.4.	Resumen de Aplicación Forense	69

Capítulo 1

Fundamentos de GNU/Linux

1.1. El origen de Linux

1.1.1. Distinción Fundamental: Kernel vs SO

Es crucial entender que Linux, no es un Sistema Operativo(SO), sino un kernel(núcleo).

- **Kernel:** Componente central del SO, desarrollado por Linus Torvalds en 1991. Es responsable de gestionar los recursos del sistema y la comunicación entre hardware y software.
- **SO:** La funcionalidad que se asocia popularmente con "Linux".^{es} la fusión del kernel de Linux con los componentes del Sistema Operativo GNU.

1.1.2. El Proyecto GNU y la Fusión con Linux

El proyecto GNU fue iniciado por Richard Stallman en 1983. El objetivo era crear un SO completo, compatible con Unix y compuesto exclusivamente de software libre. Para principios de los 90, GNU había desarrollado casi todos los componentes esenciales del SO, con la excepción del kernel. La creación del kernel de Linux por Tolvalds acabo completando el SO, creando la combinacion conocida como GNU/Linux.

1.1.3. La Filosofía del Software Libre

Las contribuciones de Richard Stallman son igualmente importantes en términos de filosofía, sentando las bases del movimiento del código abierto.

Los principios del proyecto GNU defendio la filosofía del software libre, que garantiza a los usuarios la libertad de:

- Usar el software
- Estudiar su funcionamiento
- Modificarlo
- Distribuirlo.

El impacto de esta filosofía impulso el desarrollo continuo, la profleración de Linux y el espíritu colaborativo de la comunidad de código abierto, elementos que son fundamentales para entender la evolución y la arquitectura de las distribuciones de GNU/Linux.

1.2. Arquitectura y Tipología de las Distribuciones GNU/Linux

1.2.1. Definición y Componentes de una Distribución

Una distribución de Linux es un sistema operativo funcional construido a partir del kernel de Linux más una colección de software complementario. Sus componentes esenciales incluyen:

- **Kernel de Linux:** El núcleo central del sistema.
- **Librerías Esenciales del Sistema:** Archivos necesarios para el funcionamiento de aplicaciones y procesos.
- **Gestor de paquetes:** Herramienta clave para la instalación, actualización y gestión de software. Como ejemplos tenemos Advanced Packaging Tool (APT), que se usa en Ubuntu; luego Dandified Yum(DNF) que se usa con Fedora; o Pacman que se usaría con Arch Linux.
- **Sistema de Ventanas y Entorno de Escritorio:** Proporcionan la interfaz gráfica, como X11 y Wayland para las ventanas y GNOME y KDE para el escritorio.

1.2.2. Características Clave y Relevancia Forense

- **Naturaleza Open Sourcee y Personalización:** La filosofía de código abierto permite la personalización infinita, lo que ha resultado en cientos de distros disponibles.
- **Impacto de la Fragmentación:** Esta gran variedad, si bien es una ventaja en términos de elección, resulta en una alta fragmentación. Esto implica que cada distro gestiona y almacena información (artefactos forense) de manera ligeramente diferente, lo cual es un factor crítico en cualquier investigación.
- **Distros Comunes en Entornos Empresariales:** Desde una perspectiva de informática forense, las distros que se encuentran con mayor frecuencia en entornos enterprise son Red Hat Enterprise Linux (RHEL), Ubuntu Desktop y Server, SUSE Linux.

1.2.3. El árbol genealógico y las relaciones entre distros

La mayoría de los distros modernos son derivadas (forks) de las distribuciones originales, principalmente Debian o Slackware Linux. Debian es el origen de muchos distros populares y utilizadas como Ubuntu, Mint(se basa en Ubuntu), Pop!_OS(se basa en Ubuntu) o Fedora, la cual actúa como la distribución upstream (base de desarrollo) para RHEL.

El conocimiento de estas relaciones es fundamental para que el analista forense pueda rastrear y entender las variaciones en la arquitectura del sistema y la ubicación de los datos.

1.3. WSL como Entorno Integrado para Análisis Forense

1.3.1. Evolución y Arquitectura de WSL

En 2016 Microsoft introdujo WSL, el cual es una capa de compatibilidad que permite que los binarios de Linux se ejecuten al implementar llamadas al sistema de Linux en el kernel de Windows. En 2019 se lanzó WSL 2, que era una mejora significativa. WSL 2 opera como una máquina virtual liviana que ejecuta un kernel completo de Linux, lo que significa una mayor compatibilidad con las llamadas a sistema, mejorando el soporte y el rendimiento del software. La principal ventaja de WSL 2 sobre las VMs tradicionales es su integración perfecta con el host de Windows (ej. acceso fácil al sistema de archivos de Windows y uso de Windows Terminal para gestionar múltiples shells simultáneamente)

1.3.2. Proceso de Instalación Simplificado

La instalación es sencilla, requiriendo un único comando administrativo:

```
wsl --install
```

El comando instalará la característica WSL y descarga por defecto WSL 2 con una distro inicial predeterminada de Ubuntu 22.04.4. También se requiere la creación de una cuenta de usuario de Linux separada dentro de la distro instalada.

1.3.3. Implicaciones Forenses Críticas de WSL 2

El análisis forense en entornos que utilizan WSL 2 requiere un conocimiento preciso de dónde y cómo se almacena el sistema de archivos de Linux dentro del host de Windows, ya que opera esencialmente como una máquina virtual liviana.

La pieza de evidencia más crucial es el disco virtual que contiene toda la imagen del sistema de archivos de la distribución de Linux. Por defecto, este archivo se encuentra en una ruta específica del perfil de usuario de Windows:

```
C:\Users\[Usuario]\AppData\Local\Packages\[Paquete_Distro]\LocalState\ext4.VHDX
```

El nombre del archivo, ext4.VHDX, proporciona información forense inmediata:

- **ext4:** Indica que el sistema de archivos subyacente de Linux es probable que sea ext4.
- **VHDX:** Confirma que el formato del archivo es un disco virtual Hyper-V (Virtual Hard Disk), un formato estándar de virtualización.

Una vez que se ha extraído este disco virtual, la metodología de análisis forense debe alinearse estrechamente con la investigación forense normal específica de la distro. Los artefactos forenses (como logs, historial de comandos o archivos modificados) se encontrarán en sus ubicaciones típicas de Linux, a pesar de que el sistema se ejecute en WSL 2. Incluso las técnicas de respuesta en vivo son, en gran medida, las mismas que se usarían en una instalación nativa de esa distro.

Windows facilita el acceso al sistema de archivos de WSL 2 mediante una ruta UNC integrada, lo que puede ser útil en escenarios de respuesta en vivo o investigación rápida: `\\wsl$`

Al acceder a esta ruta, se exponen las carpetas raíz de todas las distribuciones WSL instaladas. El investigador puede navegar a través de la estructura de directorios estándar de Linux (como `/boot`, `/dev`, `/etc`, `/home`, etc.) para recopilar información.

1.3.4. Archivos de Configuración Avanzados para la Investigación

La presencia y el contenido de los siguientes archivos de configuración pueden ser artefactos forenses relevantes:

- `/etc/wsl.conf`: Se encuentra dentro de la distro de Linux. Configura ajustes locales para esa distro específica.
- `%USERPROFILE%\.wslconfig`: Se encuentra en la raíz del perfil de usuario de Windows. Configura ajustes globales que afectan a todas las distros instaladas por ese usuario.

1.3.5. Artefactos del Host de Windows

El uso de WSL deja rastros forenses en el host de Windows, artefactos como Shellbags, Prefetch, Shim Cache y Amcache pueden ser perfilados para determinar la interacción del usuario con los ejecutables de WSL, como `wsl.exe` y `bash.exe`.

1.4. Appliances y Contenedores Linux

1.4.1. Appliances de Linux: Especialización y Retos Forenses

Un appliance de Linux es un software preconfigurado que se ejecuta sobre un SO Linux, optimizado y simplificado para una función específica. Estos pueden ser físicos o virtuales. A diferencia de un sistema Linux de propósito general, un appliance está optimizado para una función concreta, lo que facilita su configuración y gestión a través de una interfaz web. Para el usuario final, el SO Linux central está abstraído y oculto detrás de la GUI. El principal desafío para el investigador es obtener acceso al sistema operativo subyacente. Una vez que se obtiene acceso al SO subyacente, la metodología de investigación es la misma que la de un sistema Linux normal, ya que el appliance ejecuta una distribución de Linux. Esto significa que encontrar componentes centrales, servicios y registros estarán presentes igual que un sistema estándar de Linux, a través de `var/log`.

1.4.2. Informática Forense de Contenedores (Docker)

La proliferación de soluciones de containerización hace probable que el investigador forense deba analizar contenedores de Linux. A pesar de la abstracción, la metodología de investigación es idéntica a la de un sistema Linux normal, una vez que se accede a su sistema de archivos. Para ello vamos a ver los procesos forenses que tendríamos que realizar en los contenedores Docker:

- **1. Aislamiento:** El primer paso es aislar el contenedor para preservar su estado y evitar modificaciones (podemos usar `docker stop` o `docker pause`).
- **2. Recolección de evidencia y adquisición:**
 - **Metadatos y registros:** Extracción de metadatos(`docker inspect`) y registros en tiempo de ejecución(`docker logs`).
 - **Tráfico de red:** Captura de tráfico de red asociado al contenedor(`tcpdump`, Wireshark).
 - **Imagen forense del sistema de archivos:** Creación de una imagen forense del sistema de archivos del contenedor, que se encuentra típicamente en la ruta del host: `/var/lib/docker`.
- **3. Análisis:**
 - **Análisis del sistema de archivos:** Uso de herramientas forenses sobre la imagen para identificar archivos maliciosos o cambios.
 - **Correlación de datos:** Análisis de registros, capturas de red y cambios en archivos para reconstruir una cronología de eventos.

1.5. Estructura de Directorios

Directorio	Contenido y Función	Implicaciones Forenses
/ (Raíz)	La base de todo el sistema de archivos. Contiene todos los directorios, mount points y binarios.	Punto de inicio para la adquisición de la imagen forense.
/bin & /sbin	/bin (enlace a /usr/bin): Binarios esenciales del sistema accesibles por todos los usuarios (cp, rm, clear). /sbin (enlace a /usr/sbin): Binarios del sistema que solo deben ser ejecutados por root o mediante sudo.	Contienen las utilidades que podrían haber sido utilizadas para la actividad maliciosa o para la gestión del sistema.
/boot	Archivos necesarios para el proceso de arranque. Incluye el kernel de Linux y el sistema de archivos RAM inicial (initramfs).	Esencial para analizar la configuración de arranque y las versiones del kernel utilizado.
/dev	Contiene archivos especiales que representan dispositivos de hardware del sistema (teclado, discos duros, etc.). Refleja el concepto "todo es un archivo." ^{en} Unix.	Los dispositivos de almacenamiento se analizan a través de archivos aquí representados (ej. /dev/sda).
/etc (Etsy)	Contiene archivos de configuración del sistema y de los programas instalados (cuentas de usuario, autenticación, red, DNS).	Ubicación de artefactos críticos: hashes de contraseñas, configuraciones de servicios y de red.

/home	Contiene los directorios personales de los usuarios (excepto root). Es análogo a C:\Users en Windows.	Almacena los datos personales, historial de shell, y configuraciones específicas de cada usuario.
/lib, /lib*	Enlaces simbólicos (ej. a /usr/lib) que contienen librerías de código compartido (Shared Object files o archivos .so), similares a los DLLs de Windows.	Permiten la identificación de librerías utilizadas por programas maliciosos o modificados.
/lost+found	Presente en sistemas de archivos ext3/ext4. Almacena archivos y directorios recuperados o huérfanos por la utilidad de reparación de disco FSCK (File System Consistency Check).	Fuente potencial de datos eliminados o dañados recuperables.
/media	Punto de montaje para medios extraíbles (USB, CD/DVD) que se montan automáticamente.	Registra la interacción del sistema con dispositivos externos.
/mnt	Punto de montaje para medios persistentes o temporales (ej. discos internos permanentes o sistemas de archivos montados manualmente).	Se utiliza para el montaje temporal de evidencia o sistemas de archivos extraños.
/opt	Contiene software opcional y de terceros (ej. Splunk Universal Forwarder).	Ubicación de instalación común para software de vigilancia o aplicaciones no estándar.
/proc	Sistema de archivos virtual que contiene información sobre procesos en ejecución y datos relacionados con el kernel (ej. utilización de CPU/memoria). El contenido se genera en tiempo de ejecución.	Crítico para la respuesta en vivo (live response) al obtener información sobre el estado del sistema y los procesos activos.
/root	El directorio personal exclusivo del usuario root (superusuario).	Solo accesible con privilegios elevados; su contenido es de alta sensibilidad forense.
/run	Punto de montaje de sistema de archivos temporal (tmpfs). El contenido se almacena en RAM y se recrea en cada arranque. Utilizado por system daemons (servicios) para almacenamiento temporal y archivos de bloqueo (/run/lock).	El contenido no se preserva en una adquisición forense estática; requiere adquisición de memoria RAM para su análisis.
/srv	Contiene archivos relacionados con servicios (ej. web, FTP, rsync). Algunas distros usan /var/www para contenido web.	Puede contener archivos y configuraciones específicos de servidores en el sistema.

/sys	Sistema de archivos virtual similar a /proc. Contiene información sobre componentes del sistema y módulos del kernel (drivers).	Proporciona datos detallados del hardware y la configuración del sistema en ejecución.
/tmp	Directorio para archivos temporales del sistema, escribible por cualquier usuario (permisos rwxrwxrwt).	Un lugar común para que los actores de amenazas almacenen scripts y malware. El contenido se pierde al reiniciar si es un tmpfs o se limpia regularmente.
/usr	Contiene programas de aplicación, librerías y documentación para usuarios. Es la ruta de destino para muchos enlaces simbólicos desde la raíz.	Almacena muchos ejecutables (/usr/bin) y recursos del sistema.
/var	Contiene datos variables (variable data): registros (logs), caché y archivos spool que cambian durante la operación del sistema.	La ruta más importante para la evidencia forense estática, especialmente /var/log.

1.6. Permisos de Archivo y Directorio

La gestión de permisos en Linux es central para el modelo de seguridad y para el análisis forense. Alteraciones inusuales en permisos (especialmente los especiales) pueden indicar escaladas de privilegios o movimiento lateral.

1.6.1. Estructura de permisos

Una entrada típica se muestra como `-rwxrwxrwx:`

- **Carácter inicial:** - archivo, d directorio, l enlace simbólico.
- **Tres tríadas de permisos:**
 - Usuario propietario (u)
 - Grupo propietario (g)
 - Otros (o)

Cada posición puede tener:

- **r** (4): lectura
- **w** (2): escritura
- **x** (1): ejecución

1.6.2. Modificación de permisos (chmod)

Ejemplo: 764 = usuario rwx (7), grupo rw- (6), otros r-- (4).

Método	Base	Ejemplo	Uso forense
Numérico	r=4, w=2, x=1 → se suman	<code>chmod 764 archivo</code>	Interpretar comandos como <code>chmod 777</code> en historiales
Simbólico	u/g/o con +, -, =	<code>chmod u+x script.sh</code>	Cambios puntuales y legibles

1.6.3. Permisos especiales

Se anteponen al octal normal: 4 setuid, 2 setgid, 1 sticky. Ej.: 4755.

1.6.3.1. Set UID (4)

- **Función:** ejecuta el binario con los privilegios del propietario.
- **Riesgo:** si el propietario es `root`, cualquier usuario ejecuta con privilegios elevados.
- **Ejemplo real:** `/bin/passwd` tiene `rws` para poder modificar `/etc/shadow`.
- **Equivalente simbólico:** `chmod u+s archivo`

1.6.3.2. Set GID (2)

- **En archivos:** se ejecutan con el grupo del archivo.
- **En directorios:** los nuevos archivos heredan el grupo del directorio. Útil en carpetas compartidas.
- **Octal típico colaborativo:** `2770 dir`
- **Equivalente simbólico:** `chmod g+s dir`

1.6.3.3. Sticky bit (1)

- **Función:** en un directorio, solo el dueño del archivo, el dueño del directorio o `root` pueden borrar.
- **Uso típico:** `/tmp`
- **Equivalente simbólico:** `chmod +t dir`

1.6.4. Indicadores forenses

- `chmod 777 archivo:` intento de saltarse el control de acceso.
- Aparición de `s` en binarios que no deberían tenerla: posible escalada.
- Directorios públicos sin sticky: riesgo de borrado malicioso.
- Permisos especiales en rutas como `/usr/local/bin` o `/tmp`: revisar origen.

1.7. Gestión de Usuarios y Grupos

1.7.1. Identificadores y tipos de Usuario

- **Identificadores:** Cada usuario se identifica por un UID (User ID) y cada grupo por un GID (Group ID).
- **Usuarios del sistema:**
 - **root:** UID/GID 0. Acceso total sin restricciones.
 - **Cuentas de sistema/servicio:** UIDs y GIDs generalmente inferiores a 1000
 - **Usuarios regulares:** UIDs y GIDs generalmente a partir de los 1000

1.7.2. Archivos críticos para la investigación forense

La configuración de usuarios y grupos se distribuye en archivos de texto delimitados por dos puntos (:) ubicados en el directorio `/etc`.

1.7.2.1. `/etc/passwd`

```
login:x:UID:GID:GECOS:/home/usuario:/bin/shell
```

- **Contenido:** Almacena la identidad y la configuración básica de cada usuario.
- **Campos clave:** Nombre de usuario, UID, GID, Directorio Personal (`/home/[usuario]`) y shell predeterminado (`/bin/bash`).
- **Contraseña:** Contiene un marcador de posición x, indicando que la contraseña real se almacena en `/etc/shadow`.

1.7.2.2. `/etc/shadow`

```
usuario:hash:lastchg:min:max:warn:inactive:expire:reserved
```

- **Contenido:** Almacena la representación criptográfica (hash) de las contraseñas de los usuarios.
- **Seguridad:** Es solo legible por el usuario root debido a su alta sensibilidad.
- **Campos clave:** Nombre de usuario, hash de contraseñas, parámetros de caducidad

1.7.2.3. Algoritmos de Hashing en `/etc/shadow`

El prefijo del hash en `/etc/shadow/` identifica el algoritmo criptográfico, esencial para estimar el esfuerzo de cracking:

1.7.3. Archivos de Grupos

- `/etc/group:` Contiene los nombres de grupo, GIDs y una lista de los miembros secundarios del grupo. El grupo primario de un usuario es típicamente el que tiene el mismo nombre que su UID.
- `/etc/gshadow:` Almacena la contraseña hash del grupo, si se establece.

Algoritmo	Prefijo	Seguridad
Yescrypt	\$y\$ o \$7\$	Alto (estándar moderno, resistente a cracking offline).
SHA-512	\$6\$	Alto.
Blowfish	\$2\$, \$2a\$, \$2y\$	Medio.
MD5	\$1\$	Bajo (antiguo).

1.7.4. Utilidades de Gestión

Las siguientes utilidades son wrappers para la gestión de cuentas y se investigan en el historial de comandos de un usuario (`.bash_history`):

- **Usuarios:** `useradd`, `userdel`, `usermod`, `passwd`, `chsh`.
- **Grupos:** `groupadd`, `groupdel`, `groupmod`, `gpasswd`.

1.8. Interfaz de línea de comandos (Shells) y Artefactos de Historial

EL shell de Linux actúa como un intérprete de comandos entre el usuario y el kernel. El análisis de los shells es crucial en la informática forense, ya que son la principal fuente de evidencia de la actividad del atacante o del usuario a través del historial de comandos.

1.8.1. Tipos de Shells y Archivos de Historial

Shell	Base	Uso Principal	Archivo Historial	Notas Clave
Bourne Shell (sh)	Propio	Scripting, simple y portable.	No mantiene historial	Fundamental, aunque anticuado.
Bash (Born Again Shell)	Extiende sh	El más frecuente hoy, uso interactivo.	<code>.bash_history</code>	No registra sellos de tiempo por defecto.
Zshell (zsh)	Extiende sh/bash	Altamente personalizable.	<code>.zsh_history</code>	Requiere configuración para sellos de tiempo.

1.8.2. Archivos de configuración críticos (bash)

Estos archivos determinan el comportamiento del shell y son un objetivo para los actores de amenazas.

- `.bashrc`: Ejecutado para sesiones interactivas no de inicio de sesión.

- `.bash_profile`(o `.bash_login`): Ejecutado para sesiones de inicio de sesión.

Configuración Forense Esencial (Timestamps): La variable de entorno `HISTTIMEFORMAT` debe establecerse en estos archivos de configuración para registrar la fecha y hora de ejecución de cada comando en el historial. Esta configuración debe ser aplicada preventivamente por las organizaciones, ya que no es retroactiva.

1.8.3. Técnicas Anti-forenses

El historial de comandos es altamente susceptible a la manipulación. El investigador forense debe buscar las siguientes técnicas:

- **Eliminación o desactivación:** Búsqueda de evidencia de la eliminación de `.bash_history` o la desconfiguración de la variable `HISTFILE`.
- **Evasión del historial:** Comandos precedidos por un espacio, lo que evita que se escriban en el archivo de historial.
- **Persistencia:** Inyección de comandos para deshabilitar permanentemente el registro de historial directamente en archivos de configuración como `.bashrc`.

Valor forense: A pesar de su manipulación, el historial y los archivos de configuración son a menudo el primer punto de inicio de una investigación para reconstruir la cronología de eventos y el comportamiento del usuario/atacante.

Capítulo 2

Registro de Eventos, Logs y Monitorización

2.1. Autenticación y Seguridad

2.1.1. Fundamentos y Marco de Autenticación (PAM)

Los logs de autenticación y seguridad en Linux son críticos para administradores y para la investigación forense: permiten monitorear actividad, diagnosticar problemas y asegurar el sistema. La mayoría de estos logs se almacenan típicamente en `/var/log`.

El marco que gestiona la autenticación en Linux es el sistema de Módulos de Autenticación Conectable (PAM). PAM es un conjunto de bibliotecas compartidas que proporciona una estructura flexible para integrar múltiples tecnologías de autenticación, incluyendo contraseñas tradicionales de Unix, tokens de seguridad RSA, claves físicas (USB), y mecanismos de autenticación biométrica o de doble factor.

Archivos de Configuración de PAM

1. Configuración Principal

- Los archivos de configuración de PAM se encuentran en `/etc/pam.d/`, donde cada servicio define sus políticas de autenticación.
- Ejemplo: `common-password` define criterios globales de contraseñas, incluyendo el algoritmo de hashing usado en `/etc/shadow` (por ejemplo `yescrypt` en sistemas modernos).
- El módulo `pam_unix.so` es fundamental y se utiliza para la autenticación local por contraseña.

2. Configuración Suplementaria

- El directorio `/etc/security/` contiene archivos complementarios de configuración para políticas adicionales de PAM. Entre los más importantes:
 - `pwquality.conf`: Establece políticas de calidad de contraseña (longitud mínima, clases de caracteres, etc.).
 - `access.conf`: Define restricciones de acceso (por ejemplo, permitir o denegar inicios de sesión desde determinadas direcciones IP o subredes).

- `time.conf`: Establece restricciones de inicio de sesión basadas en franjas horarias.

Flujo de Autenticación (Ejemplo de Login Local)

El proceso de autenticación gestionado por PAM, usado por utilidades como `login`, `sudo`, `passwd` y `sshd`, suele seguir estos pasos:

1. El usuario introduce sus credenciales.
2. PAM carga la configuración del servicio específico desde `/etc/pam.d/` (y puede referenciar configuraciones adicionales en `/etc/security/`).
3. Se invoca el módulo PAM apropiado (por ejemplo `pam_unix.so`).
4. El módulo verifica la contraseña del usuario contra el hash almacenado en `/etc/shadow`.
5. Se determina el resultado (éxito o fracaso).
6. Si es exitoso, PAM inicializa la sesión.
7. Se registran los eventos y resultados relevantes para auditoría.
8. El proceso concluye con el usuario obteniendo o siendo denegado el acceso.

2.1.2. Logs Clave de Autenticación Basados en Texto

A continuación se presentan los logs de texto más relevantes relacionados con autenticación y seguridad.

Nombre del Log	Distribuciones Asociadas	Contenido Clave	Detalles de Retención y Lectura
<code>auth.log</code>	Debian, Ubuntu y derivados	Principal log para autenticación y seguridad. Registra fallos y éxitos de autenticación local (<code>sudo</code> , <code>su</code>) y remota (<code>sshd</code>). También puede incluir eventos de <code>cron</code> y del Gnome Display Manager (GDM).	La retención por defecto suele ser de 4 semanas. Los archivos antiguos se comprimen en gzip (<code>.gz</code>). Para ver archivos comprimidos: <code>zcat /var/log/auth.log.1.gz</code> . Para buscar en comprimidos: <code>zgrep "patrón/var/log/auth.log.*.gz"</code> .
<code>secure</code>	Fedora, RHEL, CentOS Stream, Rocky Linux	Contiene información similar a <code>auth.log</code> . Registra fallos y éxitos de <code>sudo</code> , <code>su</code> y <code>sshd</code> . Incluye mensajes de GDM y de Polkit.	Retención por defecto generalmente 4 semanas; rotación y compresión por <code>logrotate</code> .

Archivo Binario	Contenido	Ubicación Típica	Comando (Sistema Vivo)
btmp	Registra intentos fallidos de login (bad logins).	/var/log/btmp	lastb (p.ej. lastb -f /var/log/btmp).
wtmp	Registro histórico de logins, logouts, reinicios y apagados.	/var/log/wtmp	last (p.ej. last -f /var/log/wtmp).
utmp	Información en tiempo real sobre usuarios actualmente logueados.	/run/utmp o /var/run/utmp	who o w.
lastlog	Información sobre el último intento de login por usuario.	/var/log/lastlog	lastlog (con opciones -t o -u).
faillog	Registra intentos fallidos de login. No soportado en RHEL 6+ sin config. adicional.	/var/log/faillog	faillog (opciones -a, -t, -u).

2.1.3. Logs Binarios y Herramientas de Análisis Forense

Algunos registros se almacenan en formato binario y no pueden leerse con `cat`. Se requieren utilidades específicas para interpretarlos.

Análisis Forense de Logs Binarios (Sistemas Apagados)

Cuando el sistema objetivo está apagado (dead box) y no se pueden ejecutar comandos nativos directamente, algunas técnicas útiles son:

1. Ejecutar `strings` contra el archivo para intentar extraer texto legible.

```
strings /ruta/a/btmp | less
```

2. Abrir el archivo en un editor hexadecimal para inspección manual (por ejemplo, `hexdump -C /ruta/a/wtmp | less`).
3. Montar un sistema de prueba con la misma arquitectura/distribución, reemplazar el archivo binario del sistema investigado en el sistema de prueba y luego ejecutar los comandos nativos (`lastb`, `last`) sobre ese archivo.

Comandos Útiles

```
# Logs de texto
sudo tail -f /var/log/auth.log          # ver logs en tiempo real (Debian/Ubuntu)
sudo zcat /var/log/auth.log.1.gz        # ver archivo rotado comprimido
sudo zgrep "failed" /var/log/auth.log.*.gz # buscar en archivos comprimidos
```

```

# Logs binarios (sistema vivo)
sudo last                # ver wtmp (historial de logins)
sudo lastb              # ver btmp (intentos fallidos)
who                    # ver usuarios conectados (utmp)
w                      # ver más info sobre usuarios conectados
sudo lastlog            # ver últimos logins por usuario
sudo faillog -a         # mostrar faillog (según soporte)

# Análisis básico en dead box
strings /ruta/a/wtmp | less
hexdump -C /ruta/a/btmp | less

# Reemplazo en sistema de prueba
# Copiar /ruta/del/archivo_investigado a /var/log/wtmp en sistema de prueba y ejecuta
last -f /var/log/wtmp

```

Notas y Buenas Prácticas

- **Rotación y retención:** `logrotate` suele encargarse de la rotación y compresión de logs; los intervalos y cantidad de versiones retenidas dependen de la configuración en `/etc/logrotate.d/`.
- **Permisos y confidencialidad:** los archivos en `/var/log` contienen información sensible; mantener permisos adecuados y restricciones de acceso.
- **Auditoría adicional:** para requisitos legales o corporativos, habilitar sistemas de auditoría más avanzados (p. ej. `auditd`) y centralizar logs en un servidor de logs/siem.
- **Preservación de evidencia:** en análisis forense, evitar modificar el medio original; trabajar con copias forenses (imágenes) y documentar los pasos realizados.

2.2. Syslog y Kernel

Motor Central de Registro

El syslog es el motor central y un estándar de la industria utilizado para el registro de mensajes en sistemas tipo Unix, incluido Linux. Este protocolo es esencial porque permite que el hardware y el software generen y almacenen logs de manera eficiente, ya sea localmente o enviándolos a un servidor central.

Clasificación de Mensajes Syslog

Los mensajes generados a través de syslog se clasifican mediante dos criterios clave:

1. **Facilities (Categorías):** Indican la parte del sistema que generó el evento. Ejemplos comunes:
 - **kern:** Mensajes del kernel.
 - **user:** Mensajes de nivel de usuario.

- **mail:** Sistema de correo.
- **daemon:** Demonios del sistema.
- **auth:** Seguridad o autorización.

2. **Severity Levels (Niveles de Prioridad):** Definen la urgencia del mensaje, numerados del 0 al 7.

- **0 – Emergency:** El sistema está inutilizable.
- **1 – Alert:** Acción inmediata requerida.
- **2 – Critical:** Condición crítica.
- **3 – Error:** Error del sistema o aplicación.
- **4 – Warning:** Advertencia.
- **5 – Notice:** Condición normal pero significativa.
- **6 – Informational:** Información general.
- **7 – Debug:** Diagnóstico y depuración.

Variantes Modernas de Syslog

Existen implementaciones modernas del protocolo syslog, como Syslog-NG (1998) y RSyslog (2004). La mayoría de las distribuciones modernas utilizan RSyslog, ya que ofrece capacidades avanzadas:

- Transporte mediante TCP.
- Cifrado de mensajes.
- Filtrado basado en contenido.
- Análisis y clasificación avanzada de mensajes.

Archivos de Registro Centralizados

Estos archivos son equivalentes al Registro de Eventos del Sistema en entornos Windows.

- **/var/log/syslog (Debian/Ubuntu):**
 - Recopila un amplio rango de información del sistema, incluyendo el kernel, demonios y servicios del sistema.
 - Cada entrada incluye: marca de tiempo, nombre del host, proceso responsable, PID y mensaje.
 - Este archivo es crucial para construir líneas de tiempo forenses y obtener contexto sobre eventos específicos.
 - **Nota:** Las marcas de tiempo usan la zona horaria local, por lo que se recomienda normalizar a UTC para investigaciones multi-sistema.
- **/var/log/messages (Fedora/RHEL/Rocky Linux):**

- Equivalente funcional a `syslog`.
- Contiene información general del sistema, red y kernel.
- Solo accesible por root en muchas configuraciones.
- Fuente vital para comprender actividad del sistema y problemas de red durante un período determinado.

Registros del Kernel y Diferencias por Distribución

Aunque `syslog` o `messages` incluyen mensajes del kernel, los logs específicos del kernel ofrecen detalles más técnicos y de bajo nivel.

Sistemas Basados en Debian

1. `dmesg`

- Contiene el buffer circular (ring buffer) del kernel.
- No es persistente: solo muestra datos desde el último arranque.
- Los mensajes usan un contador en segundos desde el inicio del sistema.
- `/var/log/dmesg` guarda una copia, pero el comando `dmesg` permite consultar en vivo.

2. `/var/log/kern.log`

- Log persistente del kernel, conserva información entre reinicios.
- Incluye marcas de tiempo estándar y segundos desde el arranque.
- Ideal para análisis histórico y rastreo de errores.

Sistemas Basados en Fedora/RHEL/Rocky Linux

1. **Comando `dmesg`:** Sigue existiendo, mostrando información del buffer del kernel.
2. **Systemd Journal:** Reemplaza a `kern.log` y otros archivos tradicionales. Centraliza logs del kernel, systemd, red y servicios. Se consulta con:

```
journalctl          # Muestra todos los logs
journalctl -k       # Filtra solo los eventos del kernel
```

3. Persistencia del Journal:

- Puede ser volátil, almacenando datos en `/run/log/journal`.
- O persistente, almacenando en `/var/log/journal/`.
- Se configura en el archivo `/etc/systemd/journald.conf`.

Tipo de Log	Debian/Ubuntu	Fedora/RHEL/Rocky Linux
Registro general del sistema	/var/log/syslog	/var/log/messages
Registro del kernel (persistente)	/var/log/kern.log	Systemd Journal (journalctl -k)
Buffer temporal del kernel	dmesg	dmesg
Mecanismo de logging principal	RSyslog	Systemd Journal + RSyslog

Resumen Comparativo

Interpretación Forense

El motor Syslog proporciona la infraestructura de registro, mientras que los archivos `syslog` o `messages` sirven como registro general de eventos. Los logs del kernel ofrecen información granular y cronológica esencial para análisis forense.

La diferencia entre ambos niveles puede compararse con analizar un resumen ejecutivo (`syslog/messages`) frente a una transcripción detallada (logs del kernel): ambos son útiles, pero solo los logs de bajo nivel permiten reconstruir el contexto técnico completo detrás de un fallo o evento crítico.

2.3. Servidores Web

Formatos Estándar de Logs

Apache HTTP Server utiliza principalmente dos formatos de log para las peticiones de acceso:

1. **Common Log Format (CLF):** Formato fijo que registra campos esenciales por solicitud. Incluye:
 - Host remoto (IP del cliente)
 - Resultado del Ident
 - Usuario remoto autenticado
 - Marca de tiempo
 - Línea de solicitud (método, recurso, versión HTTP)
 - Código de estado (ej: 200, 404)
 - Tamaño de la respuesta en bytes
2. **Combined Log Format (Formato Combinado):** Extiende el CLF añadiendo dos campos:
 - **Referer:** URL de la página anterior.
 - **User Agent:** Identificación del navegador o cliente.

EngineX (Nginx) usa un formato predeterminado similar al combinado de Apache, definido por la directiva `log_format`. Los campos se expresan mediante variables como `$remote_addr`, `$status`, `$http_referer` y `$http_user_agent`. También puede registrar el tiempo total de servicio con `$request_time`.

Ubicaciones de Logs y Archivos Clave

Servidor	Distribución	Ubicación Típica	Archivos de Log Clave
Apache	Fedora	/var/log/httpd	access_log (no SSL), ssl_access_log (SSL/TLS), error_log, ssl_request_log.
Apache	Ubuntu/Debian	/var/log/apache2	access.log, error.log, other_vhosts_access.log.
EngineX	Universal	/var/log/nginx	access.log (HTTP y HTTPS), error.log (errores).

Por defecto, Nginx no separa solicitudes HTTP y HTTPS ni registra detalles de TLS o cipher suites, aunque puede personalizarse.

Apache Tomcat

Apache Tomcat es un servidor de aplicaciones Java que ejecuta servlets y JSPs. Sus logs, ubicados por defecto en `/opt/tomcat/logs`, incluyen:

- **Catalina logs:** Mensajes de error y salida estándar de la aplicación.
- **Access logs:** Registros de peticiones HTTP.

Archivos de configuración clave: `server.xml`, `logging.properties`.

Análisis Forense y Caza de Amenazas

El análisis de logs de servidores web es fundamental para la caza de amenazas y la detección de signos de compromiso (como web shells). Los investigadores deben identificar patrones o anomalías sospechosas:

- **Métodos HTTP inusuales:** Solicitudes POST o GET a rutas inesperadas.
- **Acceso repetido a archivos ejecutables** (por ejemplo, `.php`) desde la misma IP.
- **Patrones de URI anormales:** Parámetros de consulta excesivos o payloads codificados (Base64).
- **Solicitudes a archivos no estándar** o con nombres ofuscados.
- **Códigos de error repetitivos:** Abundancia de 404 o 500 puede indicar scanning.
- **Tráfico anómalo:** Peticiones desde IPs maliciosas o fuera de horario habitual.
- **Anomalías en User Agent o Referer:** Cambios bruscos en los patrones asociados a escáneres automáticos.
- **Subidas de archivos sospechosas:** Intentos de cargar archivos con dobles extensiones (`image.jpg.php`).

La rotación de logs es vital: crea copias históricas (con sufijos `.1`, `.gz`, etc.) que permiten extender el rango temporal de análisis.

2.4. Firewalls and Proxies

La Evolución del Firewall en Linux

1. **IP Chains (Precursor):** Introducido con el kernel Linux 2.2, fue el primer intento de ofrecer filtrado de paquetes en Linux. Su configuración era completamente manual, carecía de persistencia automática y no contaba con un archivo estandarizado. El logging era básico, redirigiendo los mensajes a `/var/log/messages` o `/var/log/kernel`.
2. **IP Tables (Estándar):** Introducido con el kernel 2.4, reemplazó a IP chains y continúa siendo ampliamente utilizado. Ofrece inspección de paquetes con estado y NAT. Interactúa con el framework Netfilter del kernel. Las reglas se guardan con `iptables-save`, normalmente en `/etc/iptables/rules.v4` y `/etc/iptables/rules.v6`. El logging puede configurarse por regla y suele almacenarse en `/var/log/kern.log` o `/var/log/messages`.
3. **NF Tables (El Reemplazo Moderno):** Es el sucesor de IP tables, diseñado para integrarse directamente con el kernel. Mejora el rendimiento y la flexibilidad. Los logs generados por NFTables se integran en el *journal* del sistema y se pueden visualizar con `journalctl -k` o en `/var/log/kern.log`.

Herramientas de Interfaz y Gestión Dinámica

Para simplificar el uso de IP tables y NF tables, se desarrollaron varias herramientas:

- **Shorewall:** Construido sobre IP tables. Simplifica configuraciones complejas con archivos en `/etc/shorewall`. Usa los mismos mecanismos de logging que IP tables.
- **UFW (Uncomplicated Firewall):** Interfaz simplificada para IP tables, orientada a usuarios menos técnicos. El logging se activa con `ufw logging on` y se almacena en `/var/log/ufw.log`.
- **FireHOL:** Basado en IP tables y NF tables. Su ventaja principal es una sintaxis legible para definir reglas. Configuración: `/etc/firehol/firehol.conf`.
- **Firewalld:** Solución moderna basada en zonas y servicios. Configuraciones en `/etc/firewalld/`. Logs mediante `journalctl -u firewalld`.

Prevención de Intrusiones Específicas

- **Fail2ban:** Se centra en la prevención de intrusiones. Monitorea archivos de log (autenticación SSH) y bloquea IPs tras múltiples intentos fallidos. Actualiza dinámicamente las reglas del firewall. Sus registros están en `/var/log/fail2ban.log`.

Servidores Proxy y Caching

Los servidores proxy son una capa adicional de seguridad que intermedian las solicitudes de los clientes.

- **Squid:** Servidor proxy y de caching ampliamente utilizado. Mejora la eficiencia de ancho de banda al almacenar contenido solicitado. Maneja HTTP, HTTPS y FTP. Logs: `/var/log/squid/access.log` y `/var/log/squid/cache.log`.

- **HAProxy:** Principalmente usado para balanceo de carga. Sus logs (`/var/log/haproxy.log`) ofrecen información detallada sobre el flujo.
- **EngineX (Nginx):** También actúa como proxy inverso. Los logs se almacenan en `/var/log/nginx/access.log` y `/var/log/nginx/error.log`.

Seguridad en la Contenerización

En entornos contenerizados, la seguridad depende de soluciones específicas adaptadas al tráfico interno.

- **Psyllium (Cilium):** Implementa políticas de seguridad utilizando eBPF. Ofrece microsegmentación, control granular y observabilidad. Logs en Kubernetes o en `/var/log/cilium` en el host.

Resumen Comparativo

Tecnología	Función Principal	Framework / Backend	Logging Principal
IP Chains	Filtrado básico	Kernel 2.2	<code>/var/log/messages</code>
IP Tables	Firewall con estado y NAT	Netfilter (Kernel 2.4+)	<code>/var/log/kern.log</code> , <code>/var/log/messages</code>
NF Tables	Reemplazo moderno	Kernel integrado	<code>journalctl -k</code>
UFW/ FireHOL/ Shorewall/ FirewallD	Interfaces de gestión	IP tables / NF tables	<code>/var/log/ufw.log</code> , <code>journalctl -u firewalld</code>
Fail2ban	Prevención de intrusiones	IP tables	<code>/var/log/fail2ban.log</code>
Squid/ HAProxy/ Nginx	Proxy y balanceo	Diversos	<code>/var/log/squid/</code> , <code>/var/log/haproxy.log</code> , <code>/var/log/nginx/</code>
Psyllium (Cilium)	Seguridad eBPF	Kernel eBPF	<code>/var/log/cilium</code>

2.5. Auditd

El demonio de auditoría o **auditd** es un componente fundamental del núcleo de Linux diseñado para reforzar la seguridad y el cumplimiento normativo mediante el monitoreo exhaustivo de las llamadas al sistema. Esta herramienta proporciona un registro completo de las actividades del sistema, siendo una pieza clave para la ciberseguridad moderna.

Desde una perspectiva de investigación, **auditd** ofrece un valor incalculable. Sus logs constituyen un registro detallado que documenta quién, qué, cuándo y dónde se realizó una acción. Este nivel de detalle es crucial para el análisis forense, ya que permite reconstruir la secuencia completa, identificar accesos no autorizados y determinar la causa raíz.

Ubicación y Estructura de los Logs

Los registros de `auditd` se encuentran generalmente en:

```
/var/log/audit/
```

El archivo principal es `audit.log`, el cual contiene entradas en texto plano estructurado. Un detalle técnico importante es que las marcas de tiempo se almacenan en formato epoch (Unix time), por lo que deben convertirse a un formato legible para el análisis humano.

Investigación y Análisis de Eventos

Existen dos enfoques principales:

2.5.1. Sistemas Vivos (Live Systems)

En sistemas activos, se utilizan herramientas integradas del paquete `audit`:

- **Verificar el estado del servicio:**

```
systemctl status auditd
```

- **Listar reglas activas:**

```
auditctl -l
```

Estas reglas pueden monitorear desde cambios en archivos sensibles hasta la ejecución de programas y actividad de red.

- **Buscar eventos específicos:**

```
ausearch -k <key>
```

Usa el flag `-k` para buscar por claves, y `-t recent` para acotar a eventos recientes.

2.5.2. Forensia Deadbox (Logs Recolectados)

Cuando los logs se extraen de un sistema apagado, el análisis se realiza mediante herramientas de texto:

- **Búsqueda manual con `grep`:**

```
grep "192.168.1.10" /var/log/audit/audit.log
```

Esto permite localizar eventos relacionados con una IP o un proceso específico.

- Una vez identificado un evento, se puede pivotar alrededor del mismo (por PID, UID o timestamp) para reconstruir el contexto.

2.6. Sysmon para Linux

Fundamentos y Tecnología

Sysmon para Linux es una herramienta de utilidad forense sumamente potente, considerada una adaptación directa del Sysmon para Windows. Su propósito es facilitar la investigación y análisis de entornos Linux comprometidos, proporcionando una visibilidad detallada del comportamiento del sistema.

Esta versión se basa en la librería compartida eBPF desarrollada por Microsoft CIS Internals. Aunque mantiene el mismo motor de configuración y filtrado que la versión de Windows, ha sido optimizada para capturar las particularidades del ecosistema Linux. Los principios forenses fundamentales —reconstrucción de eventos, identificación de anomalías y correlación temporal— son idénticos en ambas plataformas.

Capacidades de Registro y Enfoque Forense

La principal fortaleza de Sysmon radica en su capacidad de registrar exhaustivamente eventos críticos del sistema. Entre los eventos que captura se incluyen:

1. **Creaciones y terminaciones de procesos.**
2. **Conexiones de red y creación de sockets.**
3. **Interacciones del sistema de archivos:** creación, modificación y eliminación de archivos.
4. **Monitoreo de demonios y procesos específicos de Linux.**

Estos registros permiten a los investigadores:

- Correlacionar eventos a lo largo del tiempo y entre sistemas.
- Detectar desviaciones del comportamiento normal.
- Obtener información procesable y verificable para análisis forense.

Incluso sin configuración personalizada, Sysmon registra por defecto la creación y terminación de procesos, ofreciendo visibilidad inmediata del comportamiento del sistema.

Gestión de Registros y Visualización

Las ubicaciones típicas de los registros varían según la distribución:

Distribución	Ruta de Logs
Fedora / RHEL / Rocky Linux	/var/log/messages
Debian / Ubuntu	/var/log/syslog

Sysmon escribe los eventos en formato XML, integrándolos en los logs del sistema (syslog o messages). Esto puede dificultar la lectura directa, ya que los eventos de Sysmon se mezclan con la salida estándar del sistema.

Para solventar esta limitación, se incluye la utilidad Sysmon Log View, ubicada comúnmente en:

`/opt/sysmon/sysmon_log_view`

Esta herramienta extrae y formatea los eventos XML, presentándolos en un formato legible para humanos.

Ejemplo de uso:

```
cat /var/log/syslog | /opt/sysmon/sysmon_log_view
```

Ejemplo de Evento de Creación de Proceso

Un evento de creación de proceso típico contiene:

- Marca de tiempo.
- Ruta completa de la imagen ejecutada (`/usr/bin/cat`).
- Directorio de trabajo actual.
- Usuario asociado.
- Información del proceso padre (PID, imagen, línea de comandos).

Sysmon registra incluso acciones triviales como:

- **Creación de un directorio:** `mkdir test`.
- **Creación de un archivo:** `touch ejemplo.txt`.
- **Tráfico de red:** `ping 8.8.8.8`.

Cada uno de estos eventos queda documentado en tiempo real, incluyendo tanto la creación como la finalización del proceso.

Utilidad Comparativa

Aunque Sysmon ofrece una interfaz más familiar para quienes provienen del entorno Windows, `auditd` sigue siendo una herramienta de defensa fundamental en Linux. En la práctica, la coexistencia de Sysmon y `auditd` proporciona un marco de observabilidad y registro excepcionalmente completo, cubriendo tanto la capa de llamadas al sistema como el contexto forense detallado de los procesos.

2.7. VMware ESXi and vCenter Log Forensics

La Importancia Forense de los Logs de VMware ESXi y vCenter

Los productos de VMware, específicamente ESXi (el hipervisor) y vCenter Server (el gestor de infraestructura virtual), son componentes críticos en entornos corporativos que alojan aplicaciones y datos esenciales. Por ello, los logs que generan tienen un valor forense significativo para la investigación de incidentes de seguridad y fallos operacionales.

Logs del Hipervisor VMware ESXi

VMware ESXi, el hipervisor, registra una amplia gama de eventos sobre las máquinas virtuales que gestiona, incluso cuando opera de forma independiente. Los logs principales se encuentran en el directorio:

`/var/log`

Archivos de Log Clave para Análisis Forense

- **VMkernel.log:** Registra la actividad del kernel virtual. Es esencial para investigar fallos operativos o comportamientos anómalos de las máquinas virtuales.
- **VMkwarning.log:** Contiene advertencias generadas por el kernel virtual, útiles para detectar problemas potenciales antes de que escalen.
- **Hostd.log:** Documenta las acciones realizadas en el host ESXi por el servicio de gestión, incluyendo eventos del sistema y operaciones sobre las VMs.
- **Vpxa.log:** Describe la comunicación entre el host ESXi y el servidor vCenter, registrando la actividad del agente vCenter en el host.
- **Auth.log:** Rastrear la autenticación de usuarios; es crítico para las investigaciones de control de acceso o posibles compromisos de credenciales.
- **Shell.log:** Registra los accesos al shell del ESXi, permitiendo identificar quién ejecutó comandos dentro del host.
- **Syslog.log:** Contiene mensajes generales del sistema, proporcionando una visión global de la actividad y el estado del host.

Logs del Servidor de Gestión VMware vCenter

vCenter Server centraliza la gestión de toda la infraestructura virtual. Sus logs son esenciales para comprender las acciones administrativas y los cambios realizados en los hosts o las máquinas virtuales.

Archivo Principal de Interés: `vpxd.log`

Este archivo captura la actividad central relacionada con la gestión de los hosts ESXi y las máquinas virtuales.

Ubicación según el sistema operativo:

- **Windows:** `%allusersprofile%\VMware\vCenterServer\logs\VMware-vpx\vpxd.log`
- **Linux:** `/var/log/vmware/vpxd/vpxd.log`

Consejos para el Análisis Forense

El análisis forense de logs de VMware se beneficia de que los archivos sean texto plano, lo que permite emplear utilidades estándar de análisis y filtrado.

Herramientas útiles:

```
grep -i "error" /var/log/vmware/vpxd/vpxd.log  
awk '{print $1, $2, $3, $NF}' /var/log/hostd.log  
sed -n '/2025-11-01/,/2025-11-02/p' /var/log/vmkernel.log
```

Recomendaciones Clave:

1. **Asegurar los logs:** Inicia la recolección y protección de los archivos de log inmediatamente tras detectar un incidente para evitar pérdida o manipulación de evidencia.
2. **Correlación de información:** Cruza los eventos entre los distintos logs de ESXi y vCenter para construir una secuencia cronológica coherente de los hechos.
3. **Definir el objetivo forense:** No limitar la investigación a identificar cuándo ocurrió algo, sino comprender cómo se desarrolló el incidente y qué impacto tuvo en el entorno virtualizado.

2.8. Miscellaneous

La Extensión y la Importancia de los Registros de Linux

En un sistema Linux comprometido, la cantidad de registros relevantes puede ser enorme. Los logs estándar (como `auth.log`, `syslog` o `kern.log`) solo representan una pequeña parte del panorama. Numerosas aplicaciones de terceros generan sus propios registros personalizados dentro del sistema de archivos.

Estos logs adicionales pueden ser cruciales para reconstruir la actividad del atacante o los procesos involucrados, proporcionando evidencia que no está disponible en los registros del sistema base.

Directriz Crucial para la Adquisición Forense

Durante la adquisición forense, la instrucción fundamental es capturar absolutamente todo el contenido del directorio `/var/log`. Este directorio es el corazón del sistema de registro en Linux, y cualquier exclusión puede implicar la pérdida de evidencia valiosa.

Más allá de los registros estándar, `/var/log` puede contener logs de aplicaciones, servicios, demonios o componentes instalados manualmente, los cuales pueden proporcionar pistas decisivas.

Fuentes de Registros Específicos de Alto Valor

A continuación se destacan fuentes de logs de alto valor forense, que los investigadores deben tener siempre en cuenta:

Tipo de Log	Ubicación / Comando Típico	Detalles de Valor Forense
Systemd Journal	journalctl	Contiene información detallada sobre el sistema, procesos y eventos del kernel. Fuente integral de auditoría moderna.
Servidores de Bases de Datos	/var/log/mysql/	Servidores como MySQL o MariaDB escriben logs personalizados. Permiten rastrear consultas, autenticaciones y errores.
Logs de Correo (Mail)	/var/log/mail.log, /var/log/maillog	Registra la actividad de servicios de correo, como Postfix o Sendmail. Útiles para rastrear intentos de phishing interno o spam.
Tareas Programadas (Cron Jobs)	/var/log/cron	Documenta la ejecución de tareas programadas. También pueden aparecer en syslog . Importante para detectar scripts automatizados maliciosos.
Administradores de Paquetes	Varias ubicaciones	Rastrean instalaciones, actualizaciones y eliminaciones de software. Permiten identificar cambios sospechosos o backdoors instalados.
dpkg (Debian/Ubuntu)	/var/log/dpkg.log	Registra las operaciones de instalación y eliminación de paquetes.
yum (CentOS/RHEL)	/var/log/yum.log	Detalla actualizaciones y paquetes instalados en sistemas basados en Red Hat.
dnf (Fedora/Rocky Linux)	/var/log/dnf.log	Reemplazo moderno de yum, con estructura de logging similar.

Consideraciones Forenses Finales

- **Asegurar los logs inmediatamente** al detectar un incidente, para evitar sobrescritura o pérdida.
- **Correlacionar fuentes múltiples**, combinando logs del sistema, de servicios y de aplicaciones para obtener un panorama unificado.
- **Convertir y normalizar formatos de tiempo**, especialmente cuando se trabaja con journalctl o timestamps en formato epoch.
- **Verificar la integridad** de los archivos mediante hashes antes y después de la copia forense.

Conclusión

El dominio del análisis de logs en Linux es una de las competencias más valiosas en ciberseguridad y forense digital. Familiarizarse con la estructura de **/var/log**, las fuentes

adicionales de información y las herramientas de análisis garantiza investigaciones más precisas, reproducibles y completas.

Capítulo 3

Sistemas de Archivos en Linux

3.1. El Sistema de Archivos EXT2

3.1.1. Contexto y Características Fundamentales de EXT2

El sistema de archivos EXT2, introducido en 1993, fue uno de los pioneros adoptados por el kernel de Linux. Forma parte de una familia que incluye EXT3 y EXT4.

La característica distintiva de EXT2 es su ausencia de journaling, lo que lo hace simple, confiable y especialmente útil en entornos donde se desea minimizar las operaciones de escritura.

Casos de uso relevantes:

1. **Particiones /boot:** Utilizado dentro de configuraciones LVM, donde el journaling no es necesario.
2. **Sistemas embebidos y medios flash USB:** Su simplicidad reduce la escritura en disco, extendiendo la vida útil de medios sensibles al desgaste.

Limitaciones de tamaño: Dependen del tamaño de bloque, desde aproximadamente 2 TiB (bloques de 1 KiB) hasta 32 TiB (bloques de 4 KiB).

3.1.2. Estructura Lógica y Metadatos (Valor Forense)

EXT2 se organiza en superbloques, tablas de inodos, grupos de bloques y directorios, agrupando archivos relacionados para reducir la fragmentación.

Componentes clave:

- **Superbloque:** Contiene metadatos del sistema de archivos (tamaño, estado, número de bloques).
- **Inodos (i-nodes):** Guardan metadatos de archivos y directorios: tamaño, permisos, propietario y marcas de tiempo.

3.1.3. Marcas de Tiempo: La Cronología Forense

Las marcas de tiempo en los inodos permiten reconstruir la actividad del sistema.

Marca de Tiempo	Significado	Relevancia Forense
C time	Cambio del inodo (metadatos)	Indica modificaciones en la estructura o permisos del archivo.
M time	Modificación de contenido	Refleja la última edición del archivo.
A time	Acceso al archivo	Permite identificar ejecuciones o accesos recientes.

Advertencia Crítica: Relatime

En Linux moderno, el `a time` no se actualiza continuamente debido al impacto en el rendimiento. La opción predeterminada es `relatime`, introducida en el kernel 2.6.20, que actualiza el `a time` solo si:

1. Es anterior al `m time` o `c time`.
2. Han pasado más de 24 horas desde la última actualización.

A pesar de estas limitaciones, sigue siendo un indicador valioso para rastrear ejecuciones o accesos de archivos sospechosos.

3.1.4. Herramientas Forenses Esenciales para EXT2

La falta de journaling facilita la recuperación directa de datos. Entre las herramientas clave se incluyen:

- **stat:** Muestra el estado y las marcas de tiempo de archivos.
- **debugfs:** Permite depurar, reparar y recuperar archivos eliminados.
- **TestDisk / PhotoRec:** Recuperación de particiones y archivos borrados.
- **The Sleuth Kit (TSK):** Suite forense para análisis profundo de discos e inodos.

Comandos TSK relevantes:

Comando	Función
fls	Lista archivos y directorios, mostrando los eliminados (con asterisco).
ils	Lista información detallada del inodo (marcas de tiempo y permisos).
icat	Extrae archivos por su número de inodo.
fsstat	Muestra estadísticas y estructura general del sistema de archivos.

3.2. El Sistema de Archivos EXT3

El sistema de archivos `ext3` es una evolución directa de `ext2` y conserva sus estructuras esenciales, como superbloques, iodes y directorios. La diferencia clave y más relevante para el análisis forense es la introducción del journaling, un mecanismo diseñado para proteger la integridad del sistema y acelerar su recuperación tras un fallo.

3.2.1. Importancia del Journaling en Forense

El journaling mantiene un registro de los cambios previstos, lo que asegura que incluso tras un apagado inesperado el sistema pueda volver a un estado coherente, especialmente bajo el modo de journaling de metadatos. Desde la perspectiva forense, esto afecta directamente la forma en que se reconstruyen los eventos anteriores a un incidente.

3.2.2. Modos de Operación del Journaling en ext3

ext3 soporta tres modos de journaling, cada uno con un impacto diferente en rendimiento y consistencia:

1. **Ordered (por defecto):** Solo registra metadatos. Los datos se escriben antes que los metadatos, priorizando la consistencia.
2. **Journalled:** Registra tanto metadatos como datos antes de confirmarlos. Es el modo más seguro para la integridad.
3. **Writeback:** Solo registra metadatos y permite escribir los datos después. Ofrece mayor rendimiento a cambio de un pequeño riesgo de inconsistencia.

3.2.3. Metadatos y Límites del Sistema

ext3 mantiene el uso de iodes para almacenar metadatos de archivos. Las marcas de tiempo son esenciales para establecer cronologías forenses:

- **C time:** modificación de metadatos o del iode.
- **M time:** modificación del contenido del archivo.
- **A time:** último acceso.

Tamaños máximos:

- Volumen: 32 TiB.
- Archivo individual: 2 TiB.

3.2.4. Herramientas Forenses Fundamentales

El journaling introduce complejidad adicional que exige herramientas específicas para estudiar su contenido y reconstruir eventos. Estas

- TestDisk y PhotoRec: Útiles para recuperar datos ignorando la estructura del sistema de archivos, incluso cuando el journaling oculta restos de archivos eliminados.
- debug FS: Permite explorar y extraer metadatos, complementado con la revisión del diario.
- The Sleuth Kit (TSK): Conjunto esencial de utilidades para un análisis profundo de ext3, incluyendo sus mecanismos de journaling.

Herramienta	Función clave
JLS	Lista el contenido del diario, incluyendo transacciones incompletas.
FSstat	Muestra información global del sistema de archivos y del modo de journaling.
ILS	Lista iodes y extrae metadatos detallados.
FLS	Enumera archivos y directorios, útil para localizar elementos eliminados aún recuperables.
IAT	Extrae archivos directamente por número de iode.

Utilidades TSK más relevantes

3.2.5. Desafío Principal del Análisis en ext3

El reto forense más importante es interpretar cómo el journaling gestiona la escritura y garantiza la coherencia. Usar JLS junto con FSstat permite mapear el modo de diario y reconstruir las transacciones recientes, lo cual es fundamental para determinar los eventos previos al incidente.

Dominar estas herramientas y comprender la interacción entre journaling, metadatos y estructura interna de ext3 es indispensable para cualquier análisis forense serio sobre este sistema.

3.3. El Sistema de Archivos EXT4

ext4 es actualmente el sistema de archivos predeterminado más común en la mayoría de las distribuciones de Linux, aunque existen excepciones como Fedora (BTRFS) y Red Hat Enterprise Linux (XFS), además de ser el sucesor de ext3. ext4 fue diseñado para mejorar rendimiento, fiabilidad y capacidad de almacenamiento, introduciendo características que añaden tanto oportunidades como complejidades al análisis forense.

3.3.1. Características de ext4 fundamentales para el análisis forense

1. Extents (extensiones)

Sustituyen el mapeo clásico de bloques. Un extent representa un rango contiguo de bloques, reduciendo la fragmentación y mejorando el rendimiento.

2. Delayed Allocation (asignación retrasada)

Retrasa la asignación real de bloques hasta el momento de escritura, lo que reduce la fragmentación pero complica la recuperación de archivos recientemente eliminados o de sistemas que no se apagaron correctamente.

3. Journaling con check sums

El journal incluye sumas de verificación para detectar corrupción y asegurar la integridad de las transacciones, algo útil para reconstrucciones cronológicas.

4. Timestamps de alta precisión y CR time

ext4 añade precisión de nanosegundos a los timestamps tradicionales, además del

nuevo CR time (creation time o B time), lo que permite cronologías mucho más detalladas.

3.3.2. Herramientas clave para la investigación en ext4

- **debugfs**
Permite explorar el sistema de archivos, revisar metadatos y recuperar archivos eliminados.
- **Ext4magic**
Aprovecha el journal de ext4 para recuperar archivos que no serían accesibles mediante métodos clásicos.
- **The Sleuth Kit (TSK)**
Suite que ofrece un análisis profundo de estructuras internas de ext4.

3.3.3. Comandos TSK fundamentales para ext4

1. **FSstat**
Proporciona una visión general del estado y configuración del sistema, mostrando detalles sobre extents y check sums.
2. **FLS**
Lista archivos y directorios, útil para identificar archivos asignados pero no completamente escritos debido a la asignación retrasada.
3. **Istat**
Muestra metadatos completos de un archivo a partir de su inodo, incluyendo timestamps a nivel de nanosegundos y CR time.
4. **JLS**
Lista el contenido del journal y permite reconstruir la secuencia de operaciones previas a un fallo o apagado inesperado.

Un investigador forense en ext4 típicamente seguiría una metodología comenzando con FSstat, continuando con FLS (centrándose en archivos eliminados), examinando los detalles con Istat, y finalmente utilizando JLS para obtener pistas del journal y construir la línea de tiempo de los eventos.

3.4. Análisis de Sistemas de Archivos

3.4.0.1. Visualización Básica de Sellos de Tiempo

La forma más elemental de visualizar los metadatos temporales son los siguientes:

- **Comando ls:** El comando `ls -l` muestra por defecto el M time (modification time), que indica la última modificación del contenido del archivo.
- **Otros Sellos de Tiempo:** Se puede usar la sintaxis `ls -l --time=keyword` para ver otros sellos:

- **A time** (access time): Se visualiza con las palabras clave `atime`, `access` o `use`.
 - **C time** (inode change time): Se visualiza con `ctime` o `status`. Este sello se actualiza cuando se modifica el contenido del archivo o los metadatos (como el tamaño).
 - **CR time** (creation time o birth time): En algunas distribuciones, se puede ver usando `birth` o `creation`.
- **Comando `stat`**: Esta es la herramienta más utilizada para mostrar información detallada del archivo. Muestra el tamaño, bloques asignados, permisos (octal y simbólico), UID, GID, el número de inodo y los cuatro sellos de tiempo de ext4 (Access, Modify, Change y Birth).

Nota: Para que `stat` muestre el CR time (birth time) en ext4, el sistema debe cumplir con requisitos específicos: `coreutils` v8.31+, `glibc` v2.28+ y `kernel` v4.11+.

II. Debug FS: Depuración y Recuperación de Inodos

Debug FS es una herramienta integral para depurar sistemas de archivos ext3/ext4, permitiendo la interacción directa con las estructuras del sistema.

- **Modo No Interactivo:** Útil si el sistema no cumple los requisitos para que `stat` muestre el CR time. Se usa `sudo debugfs -R "stat <archivo>«dispositivo>` para obtener el CR time. Es necesario identificar primero el dispositivo asociado al sistema de archivos raíz (usando `df` o `mount`).
- **Modo Interactivo:** Permite acceder a un *prompt* donde se pueden usar comandos internos. Para ver información (con `stat` o `cat`), se puede usar la ruta del archivo o su número de inodo (obtenido con `ls -li`), el cual debe ir entre los símbolos `<` y `>` (ej. `<1234>`).
- **Recuperación de Archivos Eliminados:**
 - El comando `lsdel` o `list_deleted_inodes` lista los inodos eliminados.
 - La funcionalidad de `lsdel` es más aplicable a ext2, ya que en ext3/ext4 los bloques de datos no suelen estar disponibles una vez liberado el inodo.
 - Si se encuentra un inodo eliminado, se puede usar `dump` para escribir el contenido en una ruta específica.

III. Recuperación Avanzada para Ext3/Ext4 (Journaling)

Debido al journaling (registro de diario) en ext3/ext4, la recuperación es más compleja.

1. **Volcado del Diario (Journal Dump):** El primer paso es volcar el diario usando `debugfs`. El inodo número 8 (`<8>`) típicamente se refiere al diario.
 - Comando: `dump <8>/ruta/journal.dump`
2. **Uso de `ext4magic`:** Herramienta para recuperar datos usando el volcado del diario.
 - Comando: `sudo ext4magic <dispositivo>-j <diario.dump>`

- **Requisito fundamental:** La ubicación de destino (`-d`) debe ser un volumen diferente al analizado.
- **Parámetros útiles:** `-a` (después de una fecha), `-b` (antes de una fecha) y `-m` (intentar recuperar todos los archivos).

IV. The Sleuth Kit (TSK)

TSK es un conjunto de herramientas forenses para el análisis de imágenes de disco. Se recomienda montar una imagen raw (ext4) en modo solo lectura (`-o ro,loop`).

- **fsstat:** Proporciona un resumen de alto nivel y estadísticas del sistema de archivos (tipo, SO de origen, etc.).
- **fls:** Lista archivos y directorios. Crucial para crear líneas de tiempo.
 - Opciones: `-r` (recursivo), `-d` (solo eliminados), `-p` (ruta completa).
- **istat:** Muestra metadatos detallados de un archivo identificado por su número de inodo. En ext4, muestra sellos de tiempo con precisión de nanosegundos.
- **icat:** Recupera y muestra el contenido de un archivo saltándose la estructura de directorios (útil para archivos eliminados o sistemas dañados).
- **Análisis del Diario (jls y jcat):**
 - **jls:** Da una instantánea de las entradas y transacciones del diario.
 - **jcat:** Permite ver el contenido de un bloque específico del diario por su dirección.

3.4.0.2. Recuperación por Contenido (File Carving)

TestDisk y PhotoRec son herramientas de código abierto esenciales.

- **PhotoRec:** Especializada en recuperación, identificando más de 480 formatos.
- **File Carving (Talla de Archivos):** Técnica empleada por PhotoRec que ignora los metadatos del sistema de archivos. Escanea los datos brutos en busca de firmas (secuencias de bytes que indican inicio/fin de archivo, ej. `FFD8...` para JPEG).
- **Proceso de Recuperación:**
 - Ejecutar como `root`.
 - Seleccionar dispositivo (o *whole disk* para búsqueda exhaustiva).
 - Seleccionar/deseleccionar firmas.
 - Los resultados se guardan en directorios `recoup_dir`.
- **Análisis de Resultados:**
 - No es una ciencia exacta; los archivos pueden estar truncados o tener bytes extra.
 - Esto cambia el hash resultante, impidiendo coincidencias con hashes originales conocidos.
 - Se recomienda calcular nuevos hashes o usar `grep` para buscar patrones de interés (texto, IPs) dentro de los archivos recuperados.

3.5. Timestomping

El "Timestomping" (manipulación de tiempos) es una técnica de anti-forensia utilizada por actores de amenazas para evadir la detección en Windows y Linux. La esencia de esta técnica radica en antedatar un archivo para que su línea de tiempo parezca coherente con otros archivos antiguos del sistema. Nos centraremos en las implementaciones y la detección en Linux.

3.5.0.1. Ataque Mediante el Comando touch

El comando `touch` en Linux es la herramienta principal para la manipulación de tiempos.

1. Manipulación Directa:

- El atacante altera directamente el A time (acceso) y el M time (modificación) usando el parámetro `-t`.
- **Limitaciones:** `touch` no puede modificar el C time (cambio de inodo) o el CR time (creación) en sistemas ext4. Esto deja un rastro forense potencial.
- **Rastros de Precisión:** La manipulación con `-t` a menudo resulta en que el A time y el M time presenten ceros completos en la precisión de subsegundos (nanosegundos). La presencia de ceros en ambos tiempos es un valor atípico que sugiere timestomping.

2. Referencia a Archivos Existentes:

- Uso del parámetro `-r`.
- Permite robar.º copiar el A time y M time de un archivo legítimo (ej. `/usr/bin/ssh`) y aplicarlos al archivo malicioso.
- Facilita que el archivo manipulado se integre creíblemente en el sistema.

3.5.0.2. Ataque Mediante la Modificación de la Hora del Sistema

Método de baja tecnología pero efectivo, a menudo ignorado.

1. El atacante detiene los servicios de sincronización de tiempo (NTP).
2. Cambia manualmente la fecha y hora del sistema al pasado.
3. Introduce los archivos de interés (estos heredarán la fecha anticuada).
4. Reinicia los servicios NTP, sincronizando el sistema de vuelta al presente.

3.5.0.3. Estrategias de Detección Forense

La detección se basa en buscar la ejecución de comandos y las inconsistencias de los metadatos.

1. Búsqueda de Ejecución de Comandos:

- Revisar registros de procesos (*command history*, `auditd`, `sysmon`).

- Buscar el uso de `touch` con `-t` o `-r`.
- Buscar comandos de manipulación de hora como `time`, `date` o `timedatectl`.

2. Análisis de Inconsistencias:

- Buscar valores atípicos en el conjunto de datos.
- **Indicador clave:** Tiempos de modificación que son anteriores a los tiempos de creación.

3.5.0.4. Diferencias Forenses con Windows (Contexto Adicional)

Contraste con sistemas Windows que usan NTFS:

- **Comportamiento de Copia:** En Windows NTFS, es común y legítimo que la marca de modificación aparezca antes que la de creación (el M time se preserva al copiar, pero se crea un nuevo Creation time). En Linux, esto suele indicar manipulación.
- **Cantidad de Evidencia:** Los sistemas NTFS ofrecen más evidencia forense, manteniendo hasta 20 marcas de tiempo por archivo, una cantidad significativamente mayor que en Linux.

3.6. BTRFS

El Sistema de Archivos B-Tree (BTRFS o Bree), a veces llamado ButterFS, representa un cambio fundamental (paradigm shift) en el diseño de los sistemas de archivos Linux, diferenciándose significativamente de sistemas anteriores como ext2, ext3 y ext4. Es el sistema de archivos predeterminado para el escritorio de Fedora a partir de la versión 33.

3.6.1. Características Centrales y Diseño

BTRFS incorpora características avanzadas cruciales para el análisis forense moderno.

1. Copia en Escritura (Copy on Write - COW):

- **Concepto:** Los datos nunca se sobrescriben. Los datos nuevos se escriben en una ubicación diferente y los metadatos se actualizan para apuntar a la nueva información.
- **Implicación Forense:** Puede dejar rastros de versiones anteriores de archivos, mejorando significativamente las posibilidades de recuperación de datos.

2. Instantáneas (Snapshots) y Subvolúmenes:

- **Instantáneas:** Versiones de solo lectura del sistema en un punto temporal específico.
- **Subvolúmenes:** Particiones de tamaño dinámico que comparten espacio de almacenamiento. Pueden ser instantaneados individualmente.
- **Comando clave:** `btrfs subvol list` (crucial para entender la disposición de los datos).

3. Asignación Dinámica de Inodos:

- A diferencia de ext2 (fijo), BTRFS asigna inodos dinámicamente según la necesidad.
- Elimina la preocupación por el agotamiento de inodos (inode exhaustion).

4. Capacidad de Almacenamiento:

- Admite volúmenes y archivos de hasta 16 exabytes, superando a ext2 y sucesores.

3.6.2. Metadatos Forenses y O Time

BTRFS almacena metadatos enriquecidos, incluyendo los sellos tradicionales (C/M/A time) y añadiendo uno nuevo:

- **O Time (Tiempo de Origen o Nacimiento):** Registra el momento exacto en que un archivo se crea por primera vez (característica ausente en ext2/ext3).
- **Utilidad:** Enriquece la línea de tiempo de actividad del sistema.
- **Visualización:** El comando `stat` ha sido actualizado en algunas distribuciones para mostrar esta información bajo el campo "birth".

3.6.3. Herramientas Específicas para la Investigación Forense

Debido a la estructura compleja (árboles B) y dinámica, TSK tiene soporte limitado. Se requieren herramientas específicas:

Comando/Utilidad	Función	Relevancia Forense
<code>btrfs-de</code>	Utilidad para recuperar archivos eliminados.	Más efectiva que en otros sistemas debido al principio COW (acceso a versiones previas).
<code>btrfs-find-root</code>	Encuentra la raíz del sistema de archivos B-tree.	Esencial en sistemas con jerarquías complejas de instantáneas.
<code>btrfsck</code>	Comprobador de integridad.	Útil para identificar archivos o particiones corruptas.
<code>btrfs send</code>	Exporta cambios entre instantáneas.	Permite comparar instantáneas para identificar archivos modificados (o modificaciones no autorizadas).
<code>btrfs subvol find-new</code>	Identifica archivos agregados recientemente tras identificar un subvolumen.	Ayuda a rastrear la actividad reciente del sistema.

Actualmente, TSK tiene un soporte directo limitado para BTRFS. Los investigadores deben combinar TSK con los comandos nativos mencionados arriba.

3.7. El Sistema de Archivos XFS

3.7.0.1. Definición y Contexto Histórico

El sistema de archivos XFS fue creado en 1993 por Silicon Graphics. Es reconocido mundialmente por su alto rendimiento y escalabilidad, siendo especialmente adecuado para entornos empresariales y grandes volúmenes de almacenamiento.

Desde Red Hat Enterprise Linux 7 en adelante, XFS se ha establecido como el sistema de archivos predeterminado para esta distribución y sus derivados.

II. Arquitectura y Rendimiento

XFS es un sistema de archivos de journaling de 64 bits, diseñado para manejar volúmenes y archivos de hasta 8 exabytes.

Su alto rendimiento se basa en dos características clave:

1. **Operaciones de E/S Paralelas:** Diseñado para gestionar grandes flujos de datos rápidamente.
2. **Grupos de Asignación (Allocation Groups):**
 - Similares a los subvolúmenes de Btrfs.
 - Permiten dividir y gestionar el espacio en segmentos.
 - Esta segmentación facilita el acceso simultáneo a múltiples partes del disco, lo cual es crucial para sistemas con muchos núcleos (high core count systems).

Integridad: Al igual que Btrfs, XFS utiliza un journaling robusto para metadatos, asegurando que los cambios se registren antes de escribirse en el disco, previniendo corrupción y facilitando la recuperación.

III. Aspectos Forenses Clave: Marcas de Tiempo y Recuperación

Es vital considerar la implementación de marcas de tiempo (timestamps) en XFS para el análisis forense:

- **Marcas Tradicionales:** Incluye A time (acceso), M time (modificación) y C time (cambio de metadatos).
- **CR Time (Creation/Birth Time):**
 - XFS añadió esta cuarta marca de tiempo, similar a ext4.
 - Introducida con el Kernel 3.10 y `xfsprogs` v3.2.3 (Junio 2015).
 - El acceso a la precisión de subsegundos puede requerir privilegios de root. Las herramientas estándar solo muestran esta información si soportan la llamada al sistema `statx` (disponible desde Kernel 4.11).

IV. Herramientas Específicas de XFS para Investigación

El sistema operativo proporciona herramientas nativas esenciales:

Herramienta	Función	Valor Forense
<code>xfs_repair</code>	Verificación y reparación del sistema de archivos.	Recuperación tras fallos de hardware o sistema.
<code>xfs_db</code>	Depuración avanzada.	Permite analizar la estructura, recuperar metadatos y restaurar datos manualmente.
<code>xfsdump</code> / <code>xfsrestore</code>	Copia de seguridad y restauración.	Funcionalidades de alto rendimiento para preservar evidencia.

3.7.0.2. Integración con The Sleuth Kit (TSK)

La relación de XFS con el toolkit forense estándar es mixta:

1. **Limitación Oficial:** TSK no soporta oficialmente XFS en su versión estándar.
2. **Solución Forense:** Existe un fork de TSK que sí incluye soporte para XFS, permitiendo el análisis de imágenes.

Uso de herramientas TSK (versión Fork con soporte XFS):

- `fls`: Para listar directorios y buscar anomalías.
- `istat`: Para el análisis profundo de metadatos de inodos específicos.
- `icat`: Para la extracción de archivos basándose en sus números de inodo.
- `tsk_recover`: Para la potencial restauración de datos eliminados bajo circunstancias sospechosas.

3.8. Sistema de Archivos OpenZFS

OpenZFS es una implementación moderna y potente del sistema de archivos Zetabyte (ZFS), originalmente creado por Sun Microsystems para Solaris.

En el contexto de Linux, OpenZFS destaca por ser una solución avanzada que combina dos funciones críticas en una sola arquitectura: un sistema de archivos y un administrador de volúmenes lógicos (LVM).

3.8.1. Características Arquitectónicas y de Rendimiento

- **Arquitectura de 128 bits:** Le confiere una capacidad para manejar volúmenes de datos enormemente grandes.
- **Escalabilidad:** Es considerado un sistema a prueba de futuro, con límites teóricos que exceden con creces las capacidades del hardware actual.
- **Gestión Unificada:** La integración del administrador de volúmenes y el sistema de archivos simplifica drásticamente las tareas de gestión de almacenamiento.

3.8.2. Integridad de Datos como Característica Principal

El rasgo más distintivo de ZFS es su profundo énfasis en la integridad de los datos.

- **Checksums (Sumas de Verificación):** ZFS utiliza checksums para verificar cada bloque de datos. Estos se almacenan de forma independiente a los datos, permitiendo la detección y corrección automática de la corrupción silenciosa.
- **Funciones Avanzadas:**
 - Instantáneas (Snapshots) y Clones.
 - Verificación continua de integridad (Scrubbing).
 - Reparación automática (Self-healing).
 - Compatibilidad con RAID-Z.

3.8.3. Manejo de Transacciones y Timestamps

- **Grupos de Transacciones:** Para mantener la consistencia, ZFS utiliza transaction groups para confirmar los datos de manera uniforme. Cada actualización incluye su propia marca de tiempo.
- **Marcas de Tiempo (Timestamps):**
 - Soporta los estándares Unix: C time, M time, A time.
 - CR time (Birth time): OpenZFS añade el tiempo de nacimiento/creación, alcanzando la paridad de características con sistemas modernos como ext4.

3.8.4. Uso y Comandos Comunes

Aunque OpenZFS no es el estándar en la mayoría de distribuciones Linux de escritorio, es el estándar en aplicaciones especializadas y almacenamiento empresarial.

Los comandos clave para su administración son:

Comando	Descripción	Ejemplo
<code>zfs</code>	Gestiona los sistemas de archivos.	<code>zfs list</code> (muestra sistemas de archivos y snapshots).
<code>zpool</code>	Gestiona los pools de almacenamiento.	<code>zpool status</code> (muestra el estado y salud del pool).

3.8.5. Contexto en el Ámbito Digital Forense

- **Frecuencia:** A pesar de su potencia, la mayoría de los dispositivos Linux investigados habitualmente no utilizan ZFS.
- **Herramientas:** Las herramientas forenses comunes, como The Sleuth Kit (TSK), no soportan ZFS de forma nativa, lo que obliga al analista a utilizar herramientas específicas del sistema o montar las imágenes en un entorno compatible con ZFS para su análisis.

Capítulo 4

Técnicas de Persistencia de Malware

4.1. Persistencia de Malware y Gestión de Servicios: init.d vs systemd

Ahora analizaremos la evolución de los sistemas de inicialización en Linux y cómo los atacantes los explotan para lograr persistencia, comparando el sistema tradicional SysV init con el moderno systemd.

4.1.1. Persistencia mediante init.d (SysV Init)

El método tradicional se basa en scripts de shell imperativos ubicados en `/etc/init.d/`.

- **Estructura:** Los scripts utilizan bloques de metadatos conocidos como cabeceras LSB (Linux Standard Base) para definir dependencias (ej. `#Required-Start`) y los niveles de ejecución (run levels) en los que el servicio debe arrancar o detenerse.
- **Ejecución:**
 - El control del servicio se realiza mediante sentencias `case` (`start`, `stop`, `status`).
 - **Vector de ataque:** Un atacante puede inyectar comandos maliciosos (como `curl` para descargar payloads y ejecutarlos en segundo plano) dentro de la estrofa `start`.
- **Habilitación:** La persistencia se confirma mediante enlaces simbólicos gestionados por herramientas como `chkconfig` o `update-rc.d`, que asocian el script a los niveles de ejecución multiusuario (generalmente 2-5).

4.1.2. Persistencia mediante systemd

`systemd` reemplaza los scripts con archivos de unidad declarativos (unit files), ubicados comúnmente en `/etc/systemd/system/` para configuraciones locales o maliciosas.

Estructura de Unidad: Se divide en tres secciones claras:

1. `[Unit]`: Define dependencias.
 - Ejemplo: `After=network.target` asegura que la red esté activa antes de descargar un malware.

2. [Service]: Define el comportamiento. Directivas críticas para malware incluyen:
 - **ExecStart**: Comando de ejecución del binario.
 - **Restart=always**: Fuerza al sistema a revivir.^{el} proceso malicioso si este es terminado, dificultando la respuesta ante incidentes.
3. [Install]:
 - La directiva **WantedBy=multi-user.target** es el mecanismo estándar para asegurar que el malware se cargue automáticamente durante un arranque normal del sistema.

4.1.3. Análisis Comparativo en Forense

- **Legibilidad**: Mientras que `init.d` requiere interpretar scripts de shell complejos para entender la lógica del malware, `systemd` abstrae la complejidad, haciendo más legible la configuración del ataque.
- **Robustez**: `systemd` ofrece mecanismos de robustez (como el reinicio automático) que, irónicamente, favorecen al atacante al mantener el servicio activo.
- **Similitud Estructural**: Un ejemplo legítimo analizado, como `sysmon.service`, demuestra que tanto herramientas de seguridad como malware utilizan estas mismas estructuras (`Type=forking`, dependencias de `syslog`) para operar en segundo plano con privilegios de root.

4.2. Persistencia mediante Programación de Tareas: Systemd y Cron

Ahora vemos dos métodos fundamentales para la ejecución programada de código en sistemas Linux, utilizados tanto para administración legítima como para establecer persistencia de malware.

4.2.1. Temporizadores Systemd (Systemd Timers)

Representan la evolución moderna de las tareas programadas. A diferencia de un simple script, `systemd` desacopla la lógica en dos unidades:

- **Unidad de Servicio (.service)**: Define qué comando o script se ejecuta (por ejemplo, `ExecStart=/ruta/al/script`).
- **Unidad de Temporizador (.timer)**: Define cuándo se ejecuta mediante directivas como `OnCalendar`.

Implicaciones Forenses Críticas:

- **Ofuscación**: Aunque por convención suelen compartir el nombre base, la directiva `Unit=` permite que un temporizador active cualquier servicio con un nombre diferente, lo que puede usarse para ocultar actividades maliciosas.
- **Continuidad**: La opción `Persistent=true` asegura que las tareas se ejecuten tras un reinicio si se perdió su turno, garantizando la continuidad del ataque.

4.2.2. Tareas Cron (Cron Jobs)

Es el mecanismo heredado y estándar en Unix. Su configuración reside en archivos de texto llamados `crontab`.

- **Estructura:** Cada línea representa una tarea definida por seis campos: cinco para la temporalidad (minuto, hora, día del mes, mes, día de la semana) y uno para el comando.
- **Sintaxis:** Permite patrones complejos mediante el uso de comodines (*) para ejecución continua, o barras (/) para intervalos regulares.
- **Ejemplo de Ataque:** Un atacante podría configurar una tarea con cinco asteriscos (* * * * *) para ejecutar un script de conexión inversa o descarga cada minuto del año.

4.2.3. Artefactos Forenses y Ubicación

Para la detección de estos mecanismos, es necesario inspeccionar rutas específicas del sistema de archivos y utilizar comandos de interacción en vivo.

Ubicaciones en el Sistema de Archivos:

- **Tablas de usuario:** `/var/spool/cron/crontabs/`
- **Tareas del sistema y paquetes:** `/etc/cron.d/`
- **Directorios de periodicidad:** `/etc/cron.hourly/`, `/etc/cron.daily/`, etc.

Comandos de Interacción:

- `crontab -l`: Listar tareas.
- `crontab -e`: Editar tareas.
- `systemctl`: Para gestionar y verificar el estado de los temporizadores `systemd`.

4.3. Persistencia mediante Claves SSH

La técnica descrita aborda el uso del protocolo SSH como mecanismo de persistencia en un sistema previamente comprometido.

Partiendo de un escenario donde el atacante ya posee acceso inicial (una shell), el procedimiento se centra en configurar una autenticación basada en criptografía asimétrica para garantizar el acceso futuro sin necesidad de credenciales de usuario (contraseñas).

4.3.1. Generación de Credenciales (Lado del Atacante)

Se utiliza la herramienta `ssh-keygen` en la máquina local para generar un par de claves RSA. Esto produce dos archivos:

- `id_rsa` (**Clave Privada**): Debe permanecer protegida en el equipo del atacante.
- `id_rsa.pub` (**Clave Pública**): No requiere confidencialidad. Es la que será implantada.^{en} la víctima.

4.3.2. Implantación de la Clave (Lado de la Víctima)

En la máquina comprometida, se prepara el entorno para aceptar la nueva credencial. Este paso es crítico para la detección forense debido a los permisos específicos requeridos.

1. Directorio `.ssh`:

- Se verifica o crea el directorio oculto `.ssh` en la raíz del usuario comprometido.
- **Permisos:** Se asignan permisos 700 (`rwX-----`). Esto restringe el acceso únicamente al propietario para evitar errores de validación del servicio SSH (el servicio suele rechazar claves si el directorio tiene permisos de escritura para el grupo u otros).

2. Archivo `authorized_keys`:

- Se crea o edita este archivo dentro del directorio `.ssh`.
- **Acción:** El contenido de la clave pública del atacante (`id_rsa.pub`) se copia y pega aquí.
- **Permisos:** Se establecen permisos 600 (`rw-----`) para asegurar que solo el propietario tenga derechos de lectura y escritura.

4.3.3. Ejecución y Verificación

Una vez configurado el archivo de claves autorizadas:

- El atacante inicia conexión: `ssh usuario@victima`.
- **Resultado:** Al poseer la clave privada correspondiente a la pública alojada en el servidor, el sistema autentica la sesión automáticamente **sin solicitar contraseña**, consolidando la persistencia.

4.4. Mecanismos de Persistencia Avanzada en Sistemas Linux

La persistencia en sistemas Linux puede lograrse mediante la manipulación estratégica de archivos de configuración, scripts de inicio y reglas de gestión de eventos del sistema.

A continuación, se detallan los vectores técnicos específicos que permiten la ejecución arbitraria de código:

4.4.1. Manipulación de la Shell de Usuario (`.bashrc` y `.bash_profile`)

- **Ubicación:** Directorio `home` del usuario.
- **Disparador:** Se ejecutan automáticamente cada vez que se inicia una sesión de shell `bash`.
- **Técnica:** Inyección de código malicioso o creación de backdoors.
- **Aliases:** Es común alterar o crear alias de comandos comunes (como `ls` o `ll`) para ejecutar acciones maliciosas en segundo plano mientras el usuario cree ejecutar comandos legítimos.

4.4.2. Ejecución en el Inicio del Sistema (`/etc/rc.local`)

- **Concepto:** Análogo al `autoexec.bat` de sistemas antiguos.
- **Función:** Permite la ejecución automática de scripts con privilegios elevados durante el arranque.
- **Nota:** Aunque puede no existir por defecto en distribuciones modernas (como Ubuntu), su creación manual permite definir comandos que se lanzan en cada reinicio.

4.4.3. Persistencia Basada en Eventos de Hardware (`udev rules`)

- **Ubicación:** `/etc/udev/rules.d/`
- **Mecanismo:** Contiene reglas que disparan scripts en respuesta a eventos de hardware específicos.
- **Vector de Ataque:** Configurar una regla que ejecute un script automáticamente al conectar un dispositivo externo (ej. USB), convirtiendo la interacción física en un vector de ejecución.

4.4.4. Ejecución en Entornos Gráficos (XDG Autostart)

Para sistemas con interfaz gráfica (GUI), los scripts se ejecutan al cargar el entorno de escritorio.

- **Nivel Sistema:** `/etc/xdg/autostart/`
- **Nivel Usuario:** `~/.config/autostart/`

4.4.5. Scripts de Eventos de Red (NetworkManager)

- **Ubicación:** `/etc/NetworkManager/dispatcher.d/`
- **Disparador:** Scripts que se ejecutan en respuesta a cambios en el estado de la red (ej. levantar una interfaz).
- **Utilidad:** Asegura la ejecución de código condicionada a la conectividad o actividad de red.

4.4.6. Gestión de Cuentas y Privilegios

- **Creación de Usuarios:** Generar cuentas adicionales o modificar existentes para que sirvan como puertas traseras.
- **sudoers:** Alteración del archivo `/etc/sudoers` para otorgar permisos de superusuario (`root`) a cuentas comprometidas o nuevas, garantizando acceso privilegiado continuo.

Capítulo 5

Adquisición de Evidencia Digital

5.1. Adquisición de Evidencia Digital: Evolución de Herramientas

La adquisición de evidencia digital mediante la creación de imágenes de disco completo se fundamenta en el uso de herramientas de línea de comandos en entornos Unix/Linux.

Existen tres utilidades principales que han evolucionado cronológicamente para adaptarse a las necesidades de la integridad forense: `dd`, `dcfldd` y `dc3dd`.

5.1.1. La herramienta fundacional: `dd` (1974)

Es la utilidad estándar de Unix para convertir y copiar archivos a nivel de bit (bit-a-bit).

- **Funcionamiento:** Su sintaxis se basa en definir un origen (`if`), un destino (`of`) y un tamaño de bloque (`bs`).
 - *Nota:* El tamaño de bloque es crítico para el rendimiento (ej. `bs=4M` es un equilibrio habitual entre velocidad y uso de recursos).
- **Limitaciones Forenses:**
 - Carece de indicadores de progreso nativos (la terminal no muestra actividad hasta finalizar).
 - Obliga al analista a usar herramientas externas (como `watch`) para monitorizar el crecimiento del archivo.
 - No incluye funciones nativas de *hashing* o verificación durante la copia.

5.1.2. Evolución para la integridad: `dcfldd` (2002)

Desarrollada por el Laboratorio Forense del Departamento de Defensa de EE. UU. para mitigar las carencias de `dd`.

- **Mejoras:** Introduce el cálculo de hash en tiempo real (*al vuelo*) mientras se copian los datos.
- **Verificación Granular:**

- Mediante los parámetros **hash** (ej. SHA-256) y **hashwindow**, permite calcular firmas criptográficas por segmentos (ej. cada 1 GB).
- **Ventaja:** Si la imagen se corrompe parcialmente, se puede identificar qué segmento específico falló sin invalidar toda la adquisición.

5.1.3. Especialización Forense: dc3dd (2008)

Creada por el Centro de Cibercrimen del Departamento de Defensa (DC3), representa la iteración más avanzada para el análisis forense.

- **Optimización:** Elimina la necesidad de especificar manualmente el tamaño de bloque (**bs**), gestionándolo automáticamente.
- **Documentación y Logs:** Introduce parámetros dedicados para la cadena de custodia:
 - **log=:** Para el registro general de operaciones.
 - **hlog=:** Para un registro independiente de los valores hash calculados.
- **Visibilidad:** A diferencia de sus predecesores, ofrece una interfaz detallada en la salida estándar que incluye porcentaje de progreso, velocidad de transferencia y tiempo restante estimado.

Aunque **dd** sigue siendo útil por su omnipresencia en cualquier distribución Linux, **dc3dd** se establece como la opción superior para procedimientos forenses formales debido a sus capacidades automatizadas de documentación, verificación de integridad y retroalimentación visual al investigador.

5.2. Captura de Memoria Volátil en Linux: AVML

La captura de memoria volátil en investigaciones forenses es un paso crítico para obtener datos sobre el estado del sistema, los procesos activos y las conexiones de red.

5.2.1. Contexto Histórico y Limitaciones

- **Método Antiguo:** Históricamente se utilizaba el comando **dd** sobre dispositivos como **/dev/mem**.
- **Problema Actual:** Los sistemas modernos restringen el acceso a estos archivos virtuales por motivos de seguridad, haciendo inviable este método antiguo.
- **Alternativas:** Existen herramientas como **LiME** (módulo de kernel), pero **AVML** se presenta como una opción más eficiente.

5.2.2. AVML (Acquire Volatile Memory for Linux)

Es una herramienta basada en Rust y desarrollada por Microsoft.

Ventajas Técnicas:

1. **Binario Estático:** Es su principal ventaja. Permite adquirir la memoria sin necesidad de conocer la distribución específica, sin realizar compilaciones en el sistema afectado y sin requerir fingerprinting del kernel.
2. **Fuentes de Extracción:** Utiliza `/dev/crash`, `/proc/kcore` y `/dev/mem`.
3. **Compatibilidad:** Genera una salida compatible con el formato de LiME (si no se usa compresión).

5.2.3. Procedimiento Técnico

El proceso consiste en descargar el ejecutable ELF de 64 bits directamente al sistema.

1. **Permisos:** Dado que es un binario estático, solo requiere asignar permisos de ejecución (ej. `chmod 755`).
2. **Ejecución:** Debe ejecutarse con privilegios de root.
 - *Opciones:* Ofrece compresión o carga directa a Azure Blob Store.
 - *Estándar:* Simplemente especificar el nombre del archivo de salida.
3. **Verificación del Resultado:**
 - Se genera un volcado (`memory.dump`).
 - El archivo será propiedad del usuario root.
 - **Regla de Integridad:** El tamaño final debe coincidir exactamente con la cantidad de memoria RAM física del dispositivo analizado.

5.3. UAC (Unix-like Artifacts Collector)

UAC es una herramienta de respuesta ante incidentes diseñada para automatizar la recolección de evidencia forense en sistemas Linux y otras plataformas Unix (como AIX, Solaris, macOS y FreeBSD).

Su funcionamiento se basa en scripts que utilizan binarios nativos del sistema operativo, eliminando la necesidad de instalar dependencias en la máquina objetivo. Esto permite su ejecución desde dispositivos externos o recursos de red.

5.3.1. Metodología y Arquitectura

La herramienta opera respetando el orden de volatilidad de la evidencia.

Flujo de Trabajo:

1. Enumeración de herramientas disponibles (nativas o proporcionadas).
2. Carga de configuración.

3. Construcción de lista de artefactos basada en definiciones YAML y el perfil seleccionado.

Tipos de Recolectores:

- **Command:** Ejecuta comandos del sistema y guarda la salida en texto.
- **Stat:** Recolecta metadatos del sistema de archivos.
- **Find:** Localiza archivos basándose en criterios específicos.
- **Hash:** Calcula hashes de archivos y procesos.
- **File:** Adquiere archivos crudos del sistema.

5.3.2. Perfiles de Recolección

La recolección se gestiona mediante perfiles definidos en YAML, adaptando la adquisición al escenario:

- **Full:** Adquiere todos los artefactos disponibles.
- **IR Triage:** Enfocado en datos volátiles y esenciales para una respuesta rápida (excluyendo datos masivos como aplicaciones o historiales de navegador).
- **Offline:** Optimizado para imágenes de disco o sistemas no activos.

5.3.3. Estructura de la Evidencia Recolectada

El resultado se empaqueta en un archivo comprimido (`.tar.gz`) que mantiene una estructura jerárquica organizada para facilitar el análisis forense:

1. Bodyfile:

- Contiene un listado de todos los archivos del sistema en formato delimitado por tuberías (pipes).
- Compatible con herramientas como The Sleuth Kit.
- Fundamental para la creación de líneas de tiempo (timelines) del sistema de archivos.

2. Live Response:

- Almacena la salida de comandos de estado del sistema en archivos de texto.
- Incluye subdirectorios para: Hardware, Red (conexiones activas con `netstat`, `ss`), Procesos (árboles `pstree`, detalles por PID) y Configuración.

3. Root (Sistema de Archivos Reconstruido):

- UAC recrea parcialmente la estructura de directorios del sistema objetivo (ej. `/home`, `/etc`).
- Copia archivos de configuración y artefactos críticos (como `bash_history`, claves SSH y perfiles), preservando la ruta original.

Esta arquitectura permite realizar un triaje inicial exhaustivo, correlacionando procesos en memoria, conexiones de red y persistencia en disco mediante una única ejecución automatizada.

5.4. Adquisición de Evidencia Digital en Entornos Virtualizados y Cloud

La virtualización y las tecnologías en la nube permiten la adquisición de imágenes de disco y memoria directamente a nivel del hipervisor. Este enfoque ofrece la ventaja de capturar evidencia sin necesidad de interactuar directamente con el sistema operativo huésped, preservando la integridad del entorno.

5.4.1. Plataformas de Virtualización Local (On-Premise)

VMware ESXi

■ Memoria:

- Se realiza mediante una instantánea (snapshot) con la opción de memoria activada.
- **Requisito Forense:** Es imprescindible extraer tanto el archivo `.vmem` (contenido de la memoria) como el archivo `.vmsn` (que contiene los offsets necesarios para herramientas como Volatility).

■ Disco:

- La adquisición del disco completo requiere que la máquina esté apagada.
- Se debe localizar y descargar el archivo con el sufijo `-flat.vmdk`.
- Intentar copiar el archivo mientras la máquina está encendida solo resultará en un archivo descriptor de pocos kilobytes, insuficiente para el análisis.

Microsoft Hyper-V

■ Memoria:

- Se utilizan los "Puntos de control" (Checkpoints).
- **Configuración Crítica:** Cambiar de "Producción.^a Estándar" para asegurar que la información del estado de la memoria se guarde.
- Archivo resultante: `.vmrs`.

■ Disco:

- Se puede exportar la máquina virtual (encendida o apagada).
- Genera una copia del disco virtual (`.vhd` o `.vhdx`) e incluye la imagen de memoria `.vmrs` si existen checkpoints.

Proxmox VE

- A diferencia de VMware o Hyper-V, la adquisición en este hipervisor no es un proceso directo y requiere procedimientos más complejos y recursos externos específicos.

5.4.2. Entornos Cloud (Azure y AWS)

- **Adquisición de Memoria:**

- Tanto en Azure como en AWS EC2, la captura directa desde el hipervisor no siempre es accesible de forma nativa.
- **Recomendación:** Ejecutar herramientas dentro del sistema operativo huésped, como Microsoft AVML.

- **Adquisición de Disco en Azure:**

- Se debe crear una instantánea (snapshot) del disco configurada como "Full" (completa).
- Posteriormente, se exporta esta instantánea generando una URL de descarga para obtener el archivo .vhd.

- **Adquisición de Disco en AWS EC2:**

- El proceso es menos directo que en Azure y requiere pasos adicionales.

5.4.3. Estrategias de Contingencia y Análisis en Vivo

En escenarios donde el acceso a nivel de hipervisor no es factible:

1. **Herramientas Tradicionales:** Se mantiene la validez del uso de herramientas (dd, dc3dd, AVML) ejecutadas dentro de la máquina virtual huésped.
2. **Análisis en Vivo:** Para investigaciones que requieren capturar el estado operativo, es posible realizar recolecciones de triaje y análisis forense directamente sobre la VM en funcionamiento.

Capítulo 6

Líneas de Tiempo Forenses

6.1. Creación y Análisis de Líneas de Tiempo Forenses

La creación y análisis de una línea de tiempo (timeline) del sistema de archivos se realiza mediante la ejecución secuencial de herramientas de The Sleuth Kit (TSK) y software de análisis tabular especializado.

6.1.1. Generación del Archivo Intermedio (fls)

El proceso técnico comienza con la herramienta `fls`, utilizada para recorrer recursivamente el sistema de archivos de una imagen de disco.

- **Objetivo:** Generar un archivo intermedio conocido como body file.
- **Sintaxis Clave:**
 - `-r`: Para la recursividad desde el directorio raíz.
 - `-m`: Para formatear la salida en un estilo compatible con `mactime`.

6.1.2. Procesamiento y Normalización (mactime)

Posteriormente, el body file se procesa con la herramienta `mactime`.

- **Formato:** Se utiliza la opción `-d` para generar un archivo delimitado por comas (CSV).
- **Estandarización Temporal:** Se emplea la opción `-y` para estandarizar las marcas de tiempo al formato ISO 8601 en zona horaria UTC.
 - El uso de UTC es el estándar recomendado para garantizar la integridad forense y facilitar la correlación de eventos.

6.1.3. Interpretación de Datos (Timeline Explorer)

Para la interpretación de los datos resultantes, se utiliza la herramienta Timeline Explorer debido a su capacidad para manejar grandes volúmenes de datos CSV.

Análisis de la Columna MACB: El análisis se centra en la columna MACB, que desglosa los tiempos de actividad:

- **M:** Modificación (Modification)
- **A:** Acceso (Access)
- **C:** Cambio de metadatos/inodo (Change)
- **B:** Creación/Nacimiento (Birth)

La presencia de un punto (.) en esta columna indica que dicha marca de tiempo no está poblada para ese atributo específico.

6.1.4. Flujo de Trabajo Analítico

El análisis dentro de la herramienta sigue una metodología estructurada:

1. Identificación de Archivos Eliminados:

- **deleted:** Datos potencialmente recuperables.
- **deleted-realloc:** El sistema ha marcado el espacio para reutilización (datos probablemente sobrescritos).

2. Filtrado y Búsqueda:

Se aplican filtros granulares (ej. buscar extensiones `.py`) y búsqueda de Indicadores de Compromiso (IOCs).

3. Contextualización:

Una vez localizado un evento de interés, se elimina el filtro para observar el contexto temporal inmediato. Esto permite reconstruir la secuencia de eventos anterior y posterior al incidente.

6.2. Plaso: Definición y Arquitectura

Plaso se define como un conjunto de herramientas basado en Python diseñado para la informática forense. Su objetivo principal es el procesamiento de una amplia variedad de fuentes de datos, superando las capacidades de las líneas de tiempo tradicionales del sistema de archivos.

Su arquitectura central gira en torno a la creación de "super timelines", las cuales integran diversos artefactos del sistema (registros de eventos, historiales de navegación, logs de aplicaciones) en una vista cronológica unificada.

6.2.1. Componentes Principales y Flujo de Trabajo

El funcionamiento de Plaso se articula a través de varios módulos ejecutables:

- **log2timeline:** Actúa como el motor de análisis (parser). Su función es recorrer la imagen de disco, identificar artefactos y extraer eventos con marcas de tiempo, almacenándolos en un archivo de base de datos estructurado (archivo de almacenamiento Plaso).
- **pinfo:** Herramienta de diagnóstico que extrae metadatos del archivo de almacenamiento. Ofrece estadísticas sobre los tipos de eventos procesados (ej. número de entradas de historial web) y reporta errores de análisis.
- **psort:** Se encarga del post-procesamiento. Filtra, ordena y convierte los datos crudos de la base de datos en formatos legibles (como CSV dinámicos), aplicando criterios de selección temporal o temática.
- **psteel:** Funciona como un orquestador que automatiza la ejecución secuencial de **log2timeline** y **psort**, optimizando el flujo de trabajo al realizar la extracción y generación del reporte en un solo paso.

6.2.2. Metodologías de Filtrado y Recolección

El sistema permite gestionar grandes volúmenes de datos mediante dos estrategias distintas:

1. Recolección Dirigida (Filtering at Collection):

- Se aplica durante la fase de **log2timeline**.
- Restringe qué datos ingresan a la base de datos.
- **Método:** Uso de filtros de artefactos (ej. extraer solo historial de Bash) o archivos de configuración YAML para definir rutas de inclusión/exclusión.

2. Filtrado de Salida (Filtering at Output):

- Se aplica durante la fase de **psort** sobre una base de datos ya poblada.
- **Método:** Generación de líneas de tiempo parciales basadas en "time slicing" (ventanas de tiempo alrededor de un evento crítico) o rangos de fechas específicos.
- **Ventaja:** Facilita el análisis de incidentes concretos sin volver a procesar la imagen completa.

6.2.3. Análisis Contextual y Correlación

La ventaja distintiva de las "super timelines" radica en la capacidad de correlación cruzada.

- El análisis permite vincular eventos dispares, como la creación de un archivo en el sistema de archivos con registros de navegación web (historial, caché) y logs de ejecución de comandos (**sudo**, logs de sistema).
- Proporciona contexto sobre el origen de los archivos y las acciones secuenciales de un usuario, reconstruyendo la narrativa completa más allá de los metadatos básicos.

6.2.4. Herramientas Complementarias

- **image_export:** Utilidad diseñada para la extracción forense de archivos específicos desde la imagen de disco. Se basa en nombres o criterios predefinidos, preservando la estructura de directorios original y generando hashes criptográficos para la cadena de custodia.

Capítulo 7

Análisis Forense de Memoria con Volatility

7.1. Enumeración y Análisis de Procesos en Forense de Memoria (Linux)

7.1.1. Introducción al Análisis de Memoria

El análisis forense de memoria se basa en la adquisición de una “instantánea” (**snapshot**) del estado del sistema en un momento específico.

- **Naturaleza volátil:** Dado que la memoria RAM es volátil, los datos pueden perderse por reinicios o paginación.
- **Relevancia:** La RAM es crítica para investigar la actividad del sistema en tiempo real. Los procesos son los elementos más relevantes, siendo el equivalente funcional a los archivos en el análisis de disco.

7.1.2. Herramientas de Análisis: Volatility 3

Para el análisis de sistemas Linux, se emplean plugins específicos de **Volatility 3**, identificados con el prefijo `linux.` (ej. `linux.pslist`). Estos plugins interpretan las estructuras del kernel para extraer información coherente.

7.1.3. Métodos de Enumeración de Procesos

Existen dos enfoques complementarios para listar procesos, cada uno con fortalezas técnicas distintas:

7.1.4. Análisis de Contexto y Recursos del Proceso

Una vez identificados los procesos de interés, se utilizan plugins adicionales para profundizar en su comportamiento:

- **Líneas de Comando (`linux.psaux`):** Recupera los argumentos con los que se inició el proceso, aportando contexto sobre su ejecución.

Método	Plugins	Descripción	Ventajas / Limitaciones
Listado Enlazado	<code>linux.pslist</code> , <code>linux.pstree</code>	Recorre la lista doblemente enlazada de procesos partiendo de <code>init_task</code> .	Pros: Rápido, muestra jerarquía (PPID). Contras: No detecta procesos no enlazados o terminados.
Escaneo de Memoria	<code>linux.psscan</code>	Busca patrones de estructuras <code>task_struct</code> directamente en el volcado físico.	Pros: Identifica procesos ocultos (rootkits). Contras: Alto riesgo de falsos positivos.

- **Binarios Mapeados (`linux.elfs`):** Lista los archivos ejecutables ELF mapeados en memoria, identificando el código cargado.
- **Archivos Abiertos (`linux.lsof`):** Enumera descriptors de archivos (sockets, tuberías, logs). Crucial para vincular procesos con archivos en disco.
- **Mapeo de Memoria (`linux.proc.maps`):** Proporciona información sobre los segmentos de memoria mapeados, complementando a `lsof`.

7.1.5. Metodología de Investigación Recomendada

Para un análisis eficiente y profesional, se recomienda el siguiente flujo de trabajo:

1. **Persistencia:** Redirigir la salida de los plugins a archivos de texto (ej. `pslist.txt`, `psscan.txt`) para permitir filtrados rápidos con `grep`.
2. **Técnica de Comparación ("Diffing"):** Comparar los resultados de `pslist` contra `psscan`.

Indicador de compromiso: Si un proceso aparece en el escaneo de memoria pero no en la lista enlazada, es un fuerte indicio de manipulación por **rootkits**.

3. **Análisis de Jerarquía:** Utilizar `pstree` para verificar que la cadena de ejecución (ej. `bash -i proceso_extraño`) sea legítima.

7.2. Recuperación de Historial de Comandos con Volatility 3

7.2.1. Introducción al Plugin `linux.bash`

El análisis forense de memoria en Linux permite reconstruir las acciones de un usuario o atacante mediante el plugin `linux.bash`. Su función principal es extraer el historial de comandos de Bash que aún reside en la **memoria RAM**, proporcionando una visibilidad crítica sobre la actividad interactiva en el sistema comprometido.

7.2.2. Ejecución y Gestión de Datos Temporales

Al ejecutar el plugin, se obtiene una salida estructurada que incluye el **PID (Process ID)**, el nombre del proceso y el comando ejecutado.

- **El desafío de los Timestamps:** Un aspecto técnico clave es la recuperación de marcas de tiempo.
- **Contexto de memoria:** Incluso si el sistema operativo (como en el caso del dispositivo "Yamato") no tiene habilitado el registro de tiempos en el historial de archivos en disco, `linux.bash` puede extraer referencias cronológicas de las estructuras de memoria.
- **Nota Forense:** Aunque estos tiempos pueden aparecer agrupados de forma idéntica debido a la persistencia en las estructuras de datos de la RAM, ofrecen una secuencia cronológica esencial para la investigación.

7.2.3. Correlación de Eventos e Indicadores de Compromiso (IoC)

La salida de este plugin es fundamental para identificar la **cadena de muerte** (**kill chain**) del ataque. En el análisis de incidentes, permite detectar patrones como:

Etapas del Ataque	Comandos Típicos	Propósito Forense
Descarga de Payload	<code>wget</code> , <code>curl</code>	Identifica la fuente del malware (ej. <code>checknet.sh</code>).
Elevación/Ejecución	<code>chmod 755</code> , <code>sudo</code>	Muestra la preparación de scripts maliciosos para su ejecución.
Persistencia	<code>mv</code> , <code>cp</code>	Localiza el movimiento de archivos a directorios del sistema (ej. <code>/usr/local/bin</code>).
Movimiento Lateral	<code>ssh</code> , <code>scp</code>	Identifica la instalación de servicios como OpenSSH para mantener el acceso.

7.2.4. Opciones de Filtrado y Salida Profesional

Para gestionar entornos complejos con múltiples sesiones de usuario, el plugin ofrece funcionalidades avanzadas:

- **Aislamiento por PID:** El uso del argumento `--pid` permite filtrar y extraer el historial de una única sesión de Bash, evitando el ruido de otros procesos.
- **Documentación Forense:** Los resultados pueden redirigirse a archivos de texto externos (`>historial_bash.txt`), facilitando la preservación de la evidencia, el posterior análisis con herramientas de filtrado y la inclusión en el reporte final del peritaje.

Conclusión Forense: La capacidad de `linux.bash` para recuperar comandos que podrían haber sido borrados de los archivos `.bash_history` en disco convierte a la memoria RAM en la fuente de verdad más fiable tras un incidente.

7.3. Análisis de Actividad de Red en Memoria Linux

7.3.1. El Plugin `linux.sockstat`: Más allá de `lsof`

El análisis de red en memoria volátil se centra en la identificación de objetos de **socket**. Mientras que `linux.lsof` ofrece una visión general de archivos abiertos, el plugin `linux.sockstat` de Volatility 3 proporciona una granularidad técnica superior, permitiendo reconstruir la pila de comunicaciones del sistema en el momento exacto del volcado.

Datos críticos recuperados:

- **PID:** Identificador del proceso dueño de la comunicación.
- **Protocolo:** Diferenciación entre TCP, UDP y RAW.
- **Tupla de Red:** Direcciones IP (origen/destino) y puertos.
- **Estado del Socket:** Identificación de conexiones ESTABLISHED, LISTEN o estados de cierre.

7.3.2. Metodología de Análisis y Filtrado

Debido al volumen de datos (*verbose*) que genera este plugin, la lectura secuencial es ineficiente. Se recomienda la redirección a archivos de texto para aplicar filtros tácticos:

7.3.2.1. Búsqueda por PID (Vínculo Proceso-Red)

Permite confirmar si un proceso sospechoso (identificado previamente con `pslist` o `psscan`) tiene actividad de red activa.

Ejemplo Forense: Un proceso Python malicioso (como el agente DW) puede no mostrar IPs claras en ciertos estados de memoria, pero `sockstat` puede confirmar la existencia de un socket en estado `.establecido`, validando la comunicación con el C2 (Command & Control).

7.3.2.2. Búsqueda por IP (Indicadores de Compromiso)

Rastreo de direcciones IP externas o públicamente enrutables para identificar exfiltración de datos o balizamiento (*beaconing*).

7.3.3. Automatización y Enriquecimiento con Herramientas Auxiliares

Para transformar datos brutos en inteligencia accionable, se utilizan herramientas externas como **Sharp Abus** (basada en .NET 8). Este flujo de trabajo permite:

7.3.4. Conclusión Forense

La correlación entre el PID, el estado del socket y la geolocalización de las IPs permite cerrar el círculo de la investigación, vinculando un binario específico en memoria con una infraestructura de ataque externa.

Funcionalidad	Descripción
Detección de IPs Públicas	Filtra automáticamente IPs internas y se centra en tráfico enrutable.
Desduplicación	Limpia el ruido y realiza conteos de frecuencia de conexión.
Geolocalización (GeoIP)	Integra la API de <code>ipinfo.io</code> para ubicar geográficamente el destino del tráfico malicioso.

7.4. Detección de Inyección de Código con `linux.malfind`

7.4.1. Propósito y Funcionamiento

El plugin `linux.malfind` está diseñado para localizar segmentos de código ejecutable en regiones anómalas de la memoria de un proceso. Su objetivo principal es identificar áreas donde, bajo condiciones normales, no debería residir código ejecutable, como el **Heap** (montículo) o el **Stack** (pila).

Mecanismo de detección:

- **Inspección de VMA (Virtual Memory Area):** Examina las estructuras de memoria virtual buscando regiones con permisos de ejecución que no coincidan con archivos mapeados legítimos en el disco.
- **Transiciones de Permisos:** Busca cambios sospechosos de permisos, específicamente de **Escritura (W) a Ejecución (X)**, un patrón clásico en técnicas de inyección y explotación como ROP (*Return Oriented Programming*).
- **Firmas de Shellcode:** Utiliza heurística para identificar secuencias de instrucciones comunes en cargadores maliciosos (*payloads*).

7.4.2. Análisis de la Salida (Output)

La herramienta proporciona datos técnicos precisos para iniciar la investigación de un proceso comprometido:

- **PID:** Identificador del proceso bajo sospecha.
- **Direcciones de Memoria:** Rango exacto de la región VMA anómala.
- **Protección de Memoria:** Permisos actuales (ej. `rwX`).
- **Volcado Hexadecimal (Hex Dump):** Una vista parcial del contenido y su desensamblado inicial para una inspección rápida.

7.4.3. El Desafío de los Falsos Positivos

Una de las mayores limitaciones de `malfind` es su tendencia a marcar procesos legítimos. Por ello, la validación manual es obligatoria:

Hallazgo en el Hex Dump	Interpretación Forense
Bytes nulos (00) o FF	Probable Falso Positivo . Común en buffers de datos o áreas de memoria pre-asignadas.
Código no ejecutable / Basura	Probable Falso Positivo . Datos que el plugin malinterpreta como instrucciones.
Instrucciones de Shell-code	Positivo Confirmado . Presencia de llamadas al sistema (<i>syscalls</i>) o saltos inusuales.

7.4.4. Metodología de Validación Avanzada

Cuando la interpretación directa es ambigua, el flujo forense recomendado es:

1. **Extracción:** Volcar el segmento de memoria sospechoso a un archivo binario.
2. **Ingeniería Inversa:** Utilizar herramientas externas como **IDA Pro**, **Ghidra** o **Cutter** para realizar un análisis de bajo nivel.
3. **Correlación:** Cruzar el PID con `linux.lsof` y `linux.psaux` para entender qué archivos y comandos estaban asociados al proceso en el momento de la supuesta inyección.

Regla de oro: La mera presencia de una región `rwX` no garantiza la existencia de malware; es la lógica del código contenido en ella lo que determina la amenaza.

7.5. Extracción de Datos Volátiles y Volcado de Memoria

7.5.1. Consideraciones Previas: La Dependencia de la RAM Física

La extracción de datos mediante Volatility 3 está sujeta a una limitación técnica crítica: la **presencia de los datos en la memoria física**.

- Si la información ha sido **paginada (swapping)** al disco antes de la captura de la imagen de memoria, el plugin devolverá un error.
- La efectividad del volcado depende de que las estructuras no hayan sido sobrescritas o movidas al espacio de intercambio.

7.5.2. Herramientas de Extracción (Argumento `--dump`)

Para el volcado de información se utilizan tres plugins principales, cada uno orientado a una capa distinta del análisis:

7.5.2.1. Volcado de Binarios con `linux.pslist`

Al combinar este plugin con los parámetros `--dump` y `-p [PID]`, se extrae el binario completo del proceso en ejecución.

- **Uso ideal:** Análisis integral del comportamiento de un proceso sospechoso.
- **Resultado:** Permite obtener el ejecutable tal y como residía en memoria, lo que puede diferir del archivo original en disco si ha habido inyecciones.

7.5.2.2. Volcado de Regiones Específicas con `linux.proc.maps`

Este plugin permite una cirugía más fina sobre la memoria. Mediante el argumento `--address`, se pueden volcar regiones específicas (VMA) del mapa de memoria del proceso.

- **Análisis rápido:** Incluso sin herramientas de ingeniería inversa, el uso del comando `strings` sobre estos volcados puede revelar:
 - Direcciones IP y URLs ocultas.
 - Credenciales en texto plano.
 - Rutas de archivos temporales.

7.5.2.3. Extracción de Archivos ELF

El formato **ELF (Executable and Linkable Format)** es el estándar en Linux (equivalente al PE en Windows). Este plugin se especializa en volcar ejecutables y bibliotecas compartidas (`.so`).

- **Objetivo:** Análisis de integridad. Permite verificar si las librerías del sistema han sido modificadas por un atacante.
- **Utilidad:** Fundamental para identificar funciones "gancho" (hooking) o código malicioso incrustado en bibliotecas legítimas.

7.5.3. Tabla Resumen de Metodología de Volcado

Plugin	Objetivo	Parámetro Clave	Caso de Uso
<code>linux.pslist</code>	Proceso completo	<code>--dump</code>	Análisis dinámico/estático del binario.
<code>linux.proc.maps</code>	Segmento de memoria	<code>--address</code>	Recuperación de artefactos volátiles (strings).
<code>linux.elfs</code>	Archivos ELF / Librerías	<code>--dump</code>	Verificación de integridad y detección de rootkits.

7.5.4. Flujo de Trabajo Post-Extracción

Una vez obtenidos los archivos volcados, la metodología forense estándar recomienda:

1. **Cálculo de Hashes:** Generar SHA-256 de los volcados para asegurar la cadena de custodia.
2. **Análisis de Cadenas:** Ejecutar `strings -a [archivo_volcado]` para una inspección rápida.
3. **Ingeniería Inversa:** Cargar los binarios en herramientas como **Ghidra** o **IDA Pro** para analizar el flujo de ejecución malicioso.

7.6. Análisis Avanzado: Módulos del Kernel, Entorno y Escaneo Yara

7.6.1. Inspección del Kernel con `linux.lsmmod`

El análisis a nivel de kernel es preventivo y detectivo frente a las amenazas más persistentes. El plugin `linux.lsmmod` lista los módulos (LKM - *Loadable Kernel Modules*) cargados en la memoria.

- **Atributos recuperados:** Nombre del módulo, tamaño y **dirección de carga en memoria**.
- **Aplicación Forense:** Es la herramienta principal para detectar **Rootkits** que operan en el espacio del kernel.
- **Punto de control:** Un analista debe comparar esta lista con los módulos legítimos del sistema. Cualquier módulo sin un archivo correspondiente en `/lib/modules/` o con nombres aleatorios debe considerarse un Indicador de Compromiso (IoC) crítico.

7.6.2. Contexto de Ejecución: `linux.invars` (o `envirs`)

Las variables de entorno definen cómo se comporta un proceso. Este plugin extrae los pares clave-valor asociados a cada PID.

- **Utilidad Táctica:** Permite identificar configuraciones manipuladas que facilitan la persistencia o el escalado de privilegios.
- **Vectores de Ataque Comunes:**
 - **LD_PRELOAD:** Una variable clásica utilizada para forzar la carga de bibliotecas maliciosas antes que las legítimas.
 - **PATH:** Modificaciones para ejecutar binarios troyanizados en lugar de los comandos estándar del sistema.
 - **HISTFILE:** Redirección del historial para ocultar los rastros del atacante.

7.6.3. Escaneo de Memoria Virtual: `linux.vma_yarascan`

Este plugin representa una de las capacidades más potentes de Volatility 3 para la caza de amenazas (*Threat Hunting*). Escanea las estructuras **VMA (Virtual Memory Area)** buscando patrones específicos.

- **Detección de Sigilo:** Está diseñado para encontrar regiones de memoria que han sido desvinculadas de las listas de procesos estándar o que están ocultas a las herramientas comunes del sistema.
- **Integración con Reglas Yara:**
 - Permite el uso del argumento `--yara-file`.
 - Cruza las firmas de malware conocido (reglas `.yar`) con el contenido de las áreas de memoria virtual.
- **Fortaleza:** Es capaz de revelar inyecciones de código que no poseen un archivo en disco asociado (malware *fileless*), localizando fragmentos de código malicioso directamente en la RAM.

7.6.4. Resumen de Aplicación Forense

Plugin	Nivel de Análisis	Objetivo Principal
<code>linux.lsmmod</code>	Kernel	Rootkits y persistencia profunda.
<code>linux.invars</code>	Proceso / Usuario	Contexto de ejecución y manipulación de entorno.
<code>linux.vma_yarascan</code>	Memoria RAW	Malware <i>fileless</i> e inyecciones sigilosas.

Nota Metodológica: Para un análisis exhaustivo, se recomienda ejecutar `vma_yarascan` con repositorios de reglas actualizados (como los de *Yara-Rules* o *ReversingLabs*) para cubrir el mayor espectro posible de familias de malware conocidas.